

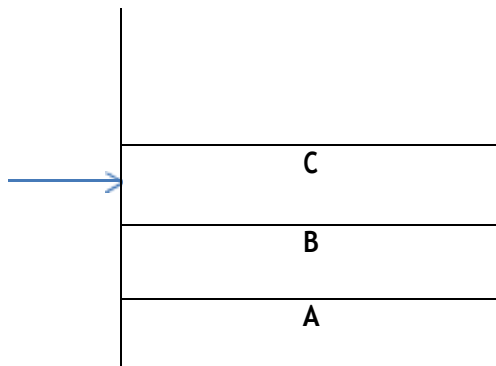
UNIT II LINEAR DATA STRUCTURES – STACKS, QUEUES 9

Stack ADT – Evaluating arithmetic expressions- other applications- Queue ADT – circular queue implementation – Double ended Queues – applications of queues

STACK

- Stack is a Linear Data Structure that follows Last In First Out(LIFO) principle.
- Insertion and deletion can be done at only one end of the stack called TOP of the stack.
- Example: - Pile of coins, stack of trays

STACK ADT:



STACK MODEL

TOP pointer

It will always point to the last element inserted in the stack.

For empty stack, top will be pointing to -1. ($TOP = -1$)

Operations on Stack (Stack ADT)

Two fundamental operations performed on the stack are PUSH and POP.

(a) PUSH:

It is the process of inserting a new element at the Top of the stack.

For every push operation:

1. Check for Full stack (overflow).
2. Increment Top by 1. ($Top = Top + 1$)

3. Insert the element X in the Top of the stack.
- (b) POP:

It is the process of deleting the Top element of the stack.

For every pop operation:

1. Check for Empty stack (underflow).
2. Delete (pop) the Top element X from the stack
3. Decrement the Top by 1. ($\text{Top} = \text{Top} - 1$)

Exceptional Conditions of stack

1. Stack Overflow

- An Attempt to insert an element X when the stack is Full, is said to be stack overflow.
- For every Push operation, we need to check this condition.

2. Stack Underflow:

- An Attempt to delete an element when the stack is empty, is said to be stack underflow.
- For every Pop operation, we need to check this condition.

12.4 Implementation of Stack

Stack can be implemented in 2 ways.

1. Static Implementation (Array implementation of Stack)
2. Dynamic Implementation (Linked List Implementation of Stack)

12.4.1 Array Implementation of Stack

- Each stack is associated with a Top pointer.
- For Empty stack, $\text{Top} = -1$.
- Stack is declared with its maximum size.

Array Declaration of Stack:

```
#define ArraySize 5
int S [ Array Size];
or
int S [ 5 ];
```

(i) Stack Empty Operation:

- Initially Stack is Empty.
- With Empty stack Top pointer points to -1 .
- It is necessary to check for Empty Stack before deleting (pop) an element from the stack.

Routine to check whether stack is empty

```
int IsEmpty ( Stack S )
{
    if( Top == - 1 )
        return(1);
}
```



(ii) Stack Full Operation:

- As we keep inserting the elements, the Stack gets filled with the elements.
- Hence it is necessary to check whether the stack is full or not before inserting a new element into the stack.

Routine to check whether a stack is full

```
int IsFull ( Stack S )
{
    if( Top == Arraysize - 1 )
        return(1);
}
```

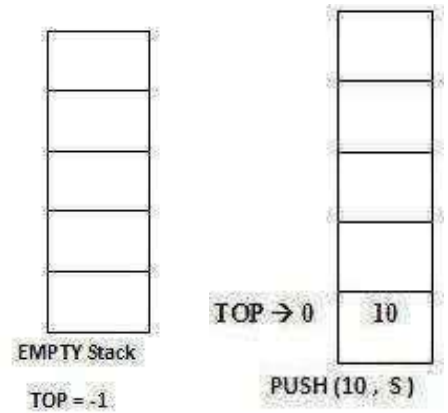


(ii) Push Operation

- It is the process of inserting a new element at the Top of the stack.
- It takes two parameters. Push(X, S) the element X to be inserted at the Top of the Stack S.
- Before inserting an Element into the stack, check for Full Stack.
- If the Stack is already Full, Insertion is not possible.
- Otherwise, Increment the Top pointer by 1 and then insert the element X at the Top of the Stack.

Routine to push an element into the stack

```
void Push ( int X , Stack S )
{
    if ( Top == Arraysize - 1)
        Error("Stack is full!!Insertion is not possible");
    else
    {
        Top = Top + 1;
        S [ Top ] =X;
    }
}
```



(iv) Pop Operation

- It is the process of deleting the Top element of the stack.
- It takes only one parameter. Pop(X).The element X to be deleted from the Top of the Stack.
- Before deleting the Top element of the stack, check for Empty Stack.
- If the Stack is Empty, deletion is not possible.
- Otherwise, delete the Top element from the Stack and then decrement the Top pointer by 1.

Routine to Pop the Top element of the stack

```
void Pop ( Stack S )
{
    if ( Top == - 1)
        Error ( "Empty stack! Deletion not possible");
    else
    {
        X = S [ Top ] ;
        Top = Top - 1 ;
    }
}
```



(v) Return Top Element

- Pop routine deletes the Top element in the stack.
- If the user needs to know the last element inserted into the stack, then the user can return the Top element of the stack.
- To do this, first check for Empty Stack.
- If the stack is empty, then there is no element in the stack.
- Otherwise, return the element which is pointed by the Top pointer in the Stack.

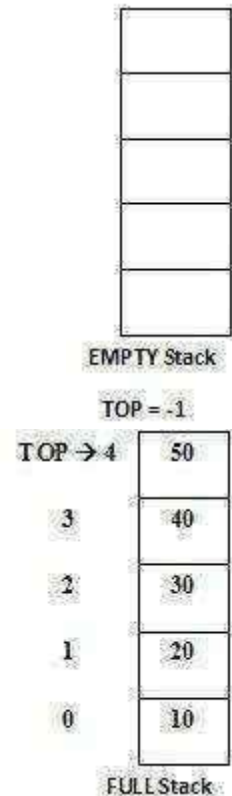
Routine to return top Element of the stack

```
int TopElement(Stack S)
{
    if(Top==-1)
    {
        Error("Empty stack!!No elements");
        return 0;
    }
    else
        return S[Top];
}
```

Implementation of stack using Array

/* static implementation of stack*/

```
#include<stdio.h>
#include<conio.h>
#define size 5
int stack [ size ];
int top;
void push( )
{
    int n ;
    printf( "\n Enter item in stack" ) ;
    scanf( " %d ", &n ) ;
    if( top == size - 1)
    {
        printf( "\nStack is Full" ) ;
    }
    else
    {
        top = top + 1 ;
```



```

        stack [ top ] = n ;
    }
}
void pop( )
{
    int item;
    if( top == - 1)
    {
        printf( "\n Stack is empty" );
    }
    else
    {
        item = stack[ top ] ;
        printf( "\n item popped is = %d" , item );
        top - -;
    }
}
void display( )
{
    int i;
    printf("\n item in stack are");
    for(i = top; i >= 0; i --)
        printf("\n %d", stack[ i ] );
}
void main( )
{
    char ch,ch1;
    ch = 'y';
    ch1 = 'y';
    top = -1;
    clrscr( );
    while(ch != 'n')
    {
        push( );
        printf("\n Do you want to push any item in stack y/n");
        ch=getch( );
    }
    display( );
    while( ch1!='n' )
    {
        printf("\n Do you want to delete any item in stack y/n");
        ch1=getch( );
        pop( );
    }
    display( );
    getch( );}

```

UTPUT:

Enter item in stack20

Do you want to push any item in stack y/n

Enter item in stack25

Do you want to push any item in stack y/n

Enter item in stack30

Stack is Full

Do you want to push any item in stack y/n

item in stack are

25

20

15

10

5

Do you want to delete any item in stack y/n

item popped is = 25

Do you want to delete any item in stack y/n

item popped is = 20

Do you want to delete any item in stack y/n

item popped is = 15

item in stack are

10

5

Linked list implementation of Stack

- Stack elements are implemented using SLL (Singly Linked List) concept.
- Dynamically, memory is allocated to each element of the stack as a node.

Type Declarations for Stack using SLL

```
struct node;
```

```
typedef struct node *stack;
```

```
typedef struct node *position;
```

```
stack S;
```

```
struct node{ int
```

```
data; position
```

```
next;};
```

```
int IsEmpty(Stack S);
```

```
void Push(int x, Stack S);
```

```
void Pop(Stack S);
```

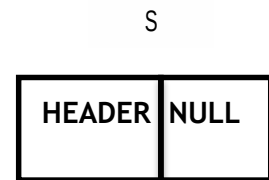
```
int TopElement(Stack S);
```

(i) Stack Empty Operation:

- Initially Stack is Empty.
- With Linked List implementation, Empty stack is represented as $S \rightarrow \text{next} = \text{NULL}$.
- It is necessary to check for Empty Stack before deleting (pop) an element from the stack.

Routine to check whether the stack is empty

```
int IsEmpty( Stack S)
{
if ( S -> next == NULL)
    return ( 1 );
}
```

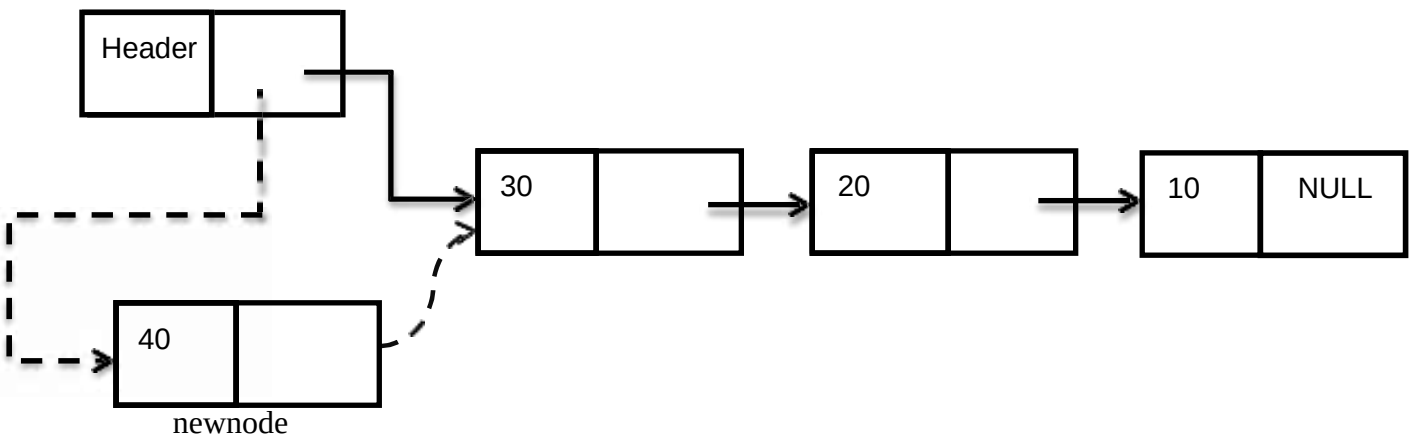


EMPTY STACK

(ii) Push Operation

- It is the process of inserting a new element at the Top of the stack.
- With Linked List implementation, a new element is always inserted at the Front of the List.(i.e.) $S \rightarrow \text{next}$.
- It takes two parameters. Push(X, S) the element X to be inserted at the Top of the StackS.
- Allocate the memory for the newnode to be inserted.
- Insert the element in the data field of the newnode.
- Update the next field of the newnode with the address of the next node which is stored in the $S \rightarrow \text{next}$.

S

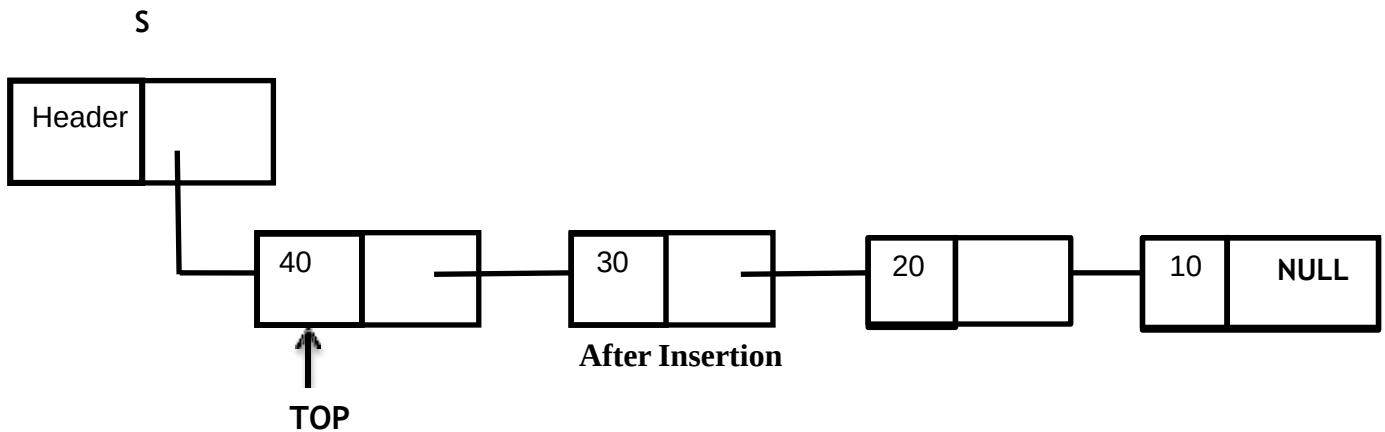


Before Insertion

Push routine

/*Inserts element at front of the list

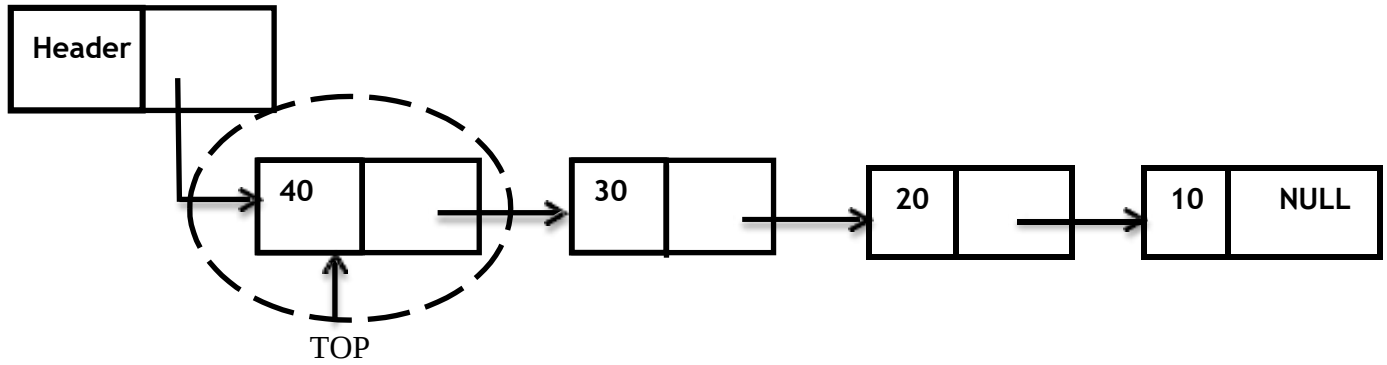
```
void push(int X, Stack S)
{
    Position newnode, Top;
    newnode = malloc (sizeof( struct node ) );
    newnode -> data = X;
    newnode -> next = S -> next;
    S -> next = newnode;
    Top = newnode;
}
```



(iii) Pop Operation

- It is the process of deleting the Top element of the stack.
- With Linked List implementations, the element at the Front of the List (i.e.) S -> next is always deleted.
- It takes only one parameter. Pop(X). The element X to be deleted from the Front of the List.
- Before deleting the front element in the list, check for Empty Stack.
- If the Stack is Empty, deletion is not possible.
- Otherwise, make the front element in the list as “temp”.
- Update the next field of header.
- Using free () function, Deallocate the memory allocated for temp node.

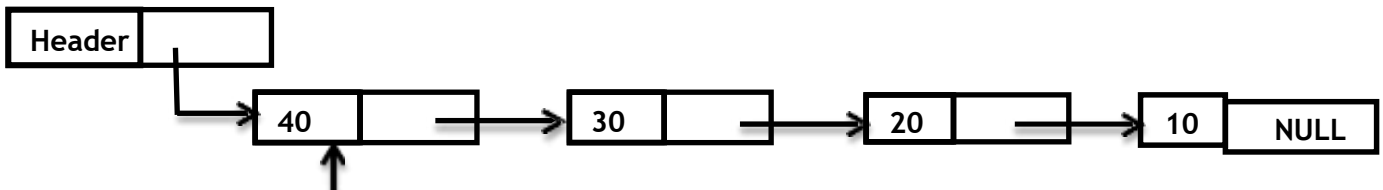
S R

**Before Deletion****Pop routine****/*Deletes the element at front of list**

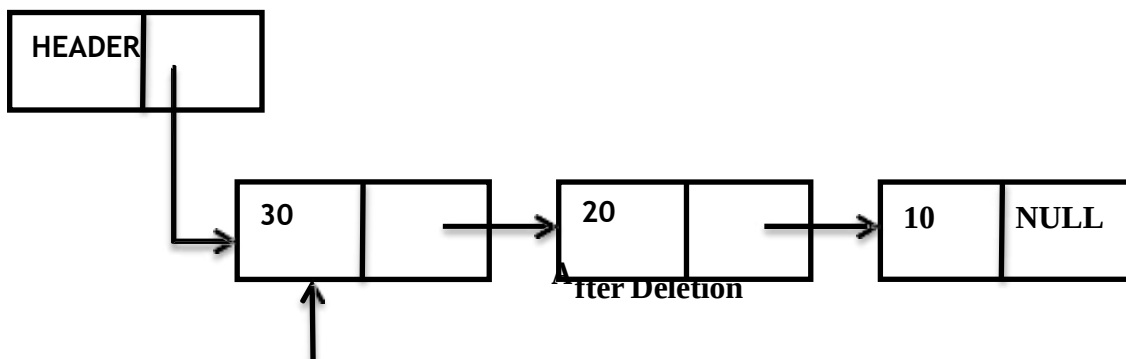
```

void Pop( Stack S )
{
    Position temp, Top;
    Top = S -> next;
    if( S -> next == NULL)
        Error("empty stack! Pop not possible");
    else
    {
        Temp = S -> next;
        S -> next = temp -> next;
        free(temp);
        Top = S -> next;
    }
}

```



S



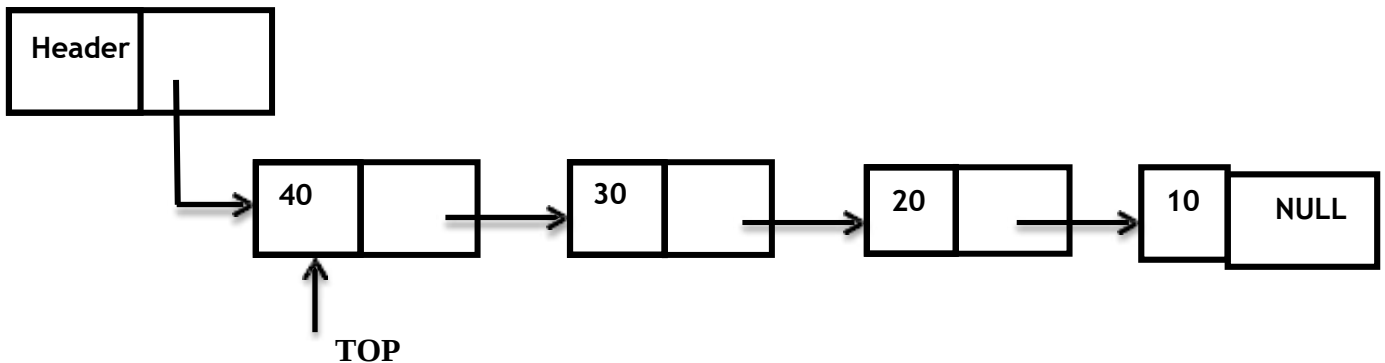
(iv) Return Top Element

- Pop routine deletes the Front element in the List.
- If the user needs to know the last element inserted into the stack, then the user can return the Top element of the stack.
- To do this, first check for Empty Stack.
- If the stack is empty, then there is no element in the stack.
- Otherwise, return the element present in the S -> next -> data in the List.

Routine to Return Top Element

```
int TopElement(Stack S)
{
    if(S->next==NULL)
    {
        error("Stack is empty");
        return 0;
    }
    else
        return S->next->data;
}
```

S



Implementation of stack using 'Linked List'

```
/* Dynamic implementation of stack*/
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
typedef struct node * position ;
struct node
{
    int data ;
    position next ;
};
void create( ) ;
void push( ) ;
void pop( ) ;
void display( ) ;
position s, newnode, temp, top ; /* Global Declarations */
void main() {
    /* Main Program */
    int op ;
    clrscr( ) ;
    do {
        printf( "\n ### Linked List Implementation of STACK Operations ### \n\n" ) ;
        printf( "\n Press 1-create\n 2-Push\n 3-Pop\n 4-Display\n5-Exit\n" ) ;
        printf( "\n Your option ? " ) ;
        scanf( " % d ", & op ) ;
        switch (op) {
            case 1:
                create( ) ;
                break ;
            case 2:
                push();
                break;
            case 3:
                pop();
                break;

            case 4:
                display();
                break;
            case 5:
                exit(0);

        }
    }while(op<5);
```

```

getch();
}
void create()
{
    int n,i;
    s=NULL;
    printf("Enter the no of nodes to be created\n");
    scanf("%d",&n);
    newnode=(struct node*)malloc(sizeof(struct node));
    printf("Enter the data\t");
    scanf("%d",&newnode->data);
    newnode->next=NULL;
    top=newnode;
    s=newnode;
    for(i=2;i<=n;i++)
    {
        newnode=(struct node*)malloc(sizeof(struct node));
        printf("Enter the data\t");
        scanf("%d",&newnode->data);
        newnode->next=top;
        s=newnode;
        top=newnode;
    }
}
void display()
{
    top=s;
    while(top!=NULL)
    {
        printf("%d->",top->data);
        top=top->next;
    }
    printf("NULL\n");
}
void push()
{
    top=s;
    newnode=(struct node*)malloc(sizeof(struct node));
    printf("Enter the data\t");
    scanf("%d",&newnode->data);
    newnode->next=top;
    top=newnode;
    s=newnode;
    display();
}
void pop()
{
    top=s;

```

```

if(top==NULL)
printf("Empty stack\n\n");
else
{
temp=top;
printf("Deleted element is \t %d\n\n",top->data);
s=top->next;
free(temp);
display();
} }

```

Output

Linked List Implementation of STACK Operations

Press 1-create

2-Push

3-Pop

4-Display

5-Exit

Your option ? 1

Enter the no of nodes to be created5

Enter the data 10

Enter the data20

Enter the data30

Enter the data40

Enter the data50

Linked List Implementation of STACK Operations

Press 1-create

2-Push

3-Pop

4-Display

5-Exit

Your option ? 4

50->40->30->20->10->NULL

Linked List Implementation of STACK Operations

Press 1-create

2-Push

3-Pop

4-Display

5-Exit

Your option ?2

Enter the data60

Your option ? 4

60->50->40->30->20->10->NULL

Your option ?2

Enter the data70

Your option ? 4

70->60->50->40->30->20->10->NULL

Your option ? 3

Deleted element is 70

Your option ? 4

50->40->30->20->10->NULL

Applications of Stack

The following are some of the applications of stack:

1. Evaluating the arithmetic expressions
 - Conversion of Infix to Postfix Expression
 - Evaluating the Postfix Expression
2. Balancing the Symbols
3. Function Call
4. Tower of Hanoi
5. 8 Queen Problem

Evaluating the Arithmetic Expression

There are 3 types of Expressions

- Infix Expression
- Postfix Expression
- Prefix Expression

INFIX:

The arithmetic operator appears between the two operands to which it is being applied.

$$A / B + C$$

POSTFIX:

The arithmetic operator appears directly after the two operands to which it applies. Also called reverse polish notation.

$$((A / B) + C)$$

$$AB/C +$$

PREFIX:

The arithmetic operator is placed before the two operands to which it applies. Also called polish notation.

$$((A/B) + C)$$

$$+ / ABC$$

Evaluating Arithmetic Expressions

1. Convert the given infix expression to Postfix expression
2. Evaluate the postfix expression using stack.

Algorithm to convert Infix Expression to Postfix Expression:

Read the infix expression one character at a time until it encounters the delimiter “#”

Step 1: If the character is an operand, place it on the output.

Step 2: If the character is an operator, push it onto the stack. If the stack operator has a higher or equal priority than input operator then pop that operator from the stack and place it onto the output.

Step 3: If the character is left parenthesis, push it onto the stack

Step 4: If the character is a right parenthesis, pop all the operators from the stack till it encounters left parenthesis, discard both the parenthesis in the output.

E.g. Consider the following Infix expression: - $A*B+(C-D/E)\#$

Read char	Stack	Output
A		A
*	*	A
B	+ *	AB
+	+	AB*
(	(+	AB*

Read char	Stack	Output
C	(+	AB*C
-	- (+	AB*C
D	- (+	AB*CD
/	/ - (+	AB*CD
E	/ - (+	AB*CDE
)	/ - (+	AB*CDE/-

Read char	Stack	Output
#		AB*CDE/-+

Output: Postfix expression:- AB*CDE/-+

Evaluating the Postfix Expression

Algorithm to evaluate the obtained Postfix Expression

Read the postfix expression one character at a time until it encounters the delimiter „#“

Step 1: If the character is an operand, push its associated value onto the stack.

Step 2: If the character is an operator, POP two values from the stack, apply the operator to them and push the result onto the stack.

E.g consider the obtained **Postfix expression:- AB*CDE/-+**

Operand	Value
A	2
B	3
C	4
D	4
E	2

Char Read	Stack
A	2
B	3 2

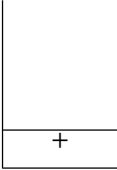
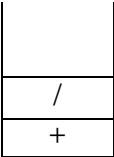
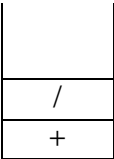
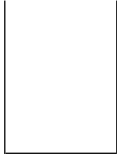
Char Read	Stack			
*	6			
C	4 6			
D	4 4 6			
E				
/	<table><tr><td>2</td></tr><tr><td>4</td></tr><tr><td>6</td></tr></table>	2	4	6
2				
4				
6				
-	<table><tr><td>2</td></tr><tr><td>6</td></tr></table>	2	6	
2				
6				
+	<table><tr><td>8</td></tr></table>	8		
8				

OUTPUT = 8

Example 2: Infix expression:- (a+b)*c/d+e/f#

Read char	Stack	Output
(	(	
a	(	a

+	+ (	a
b	+ (	ab
)		ab+
*	*	ab+
c	*	ab+c
/	/	ab+c*
d	/	ab+c*d
+	+	ab+c*d/

e		ab+c*d/e
/		ab+c*d/e
f		ab+c*d/ef
#		ab+c*d/ef/+

Postfix expression:- ab+c*d/ef/+

Evaluating the Postfix Expression

Operand	Value
a	1
b	2
c	4
d	2
e	6
f	3

Char Read	Stack
a	1
b	2 1
+	3
c	4 3
*	12
d	2 12
/	6
e	6 6
F	3 6 6
/	2 6
+	8

Output = 8

Infix to Postfix Conversion	Output
<pre> #define SIZE 50 /* Size of Stack */ #include <ctype.h> char s[SIZE]; int top=-1; /* Global declarations */ void push(char elem) { s[++top]=elem; } char pop() { return(s[top--]); } int pr(char elem) { /* Function for precedence */ switch(elem) { case '#': return 0; case '(': return 1; case '+': case '-': return 2; case '*': case '/': return 3; } return 0; } Void main() { /* Main Program */ char infx[50],pofx[50],ch,elem; int i=0,k=0; printf("\nRead the Infix Expression ? "); scanf("%s",infx); push('#'); while((ch=infx[i++]) != '\0') { if(ch == '(') push(ch); else if(isalnum(ch)) pofx[k++]=ch; else if(ch == ')') { while(s[top] != '(') </pre>	Read the Infix Expression ? (a+b)-(c-d) Given Infix Expn: (a+b)*(c-d) Postfix Expn: ab+cd-*

<pre> pofx[k++]=pop(); elem=pop(); /* Remove */ } else { /* Operator */ while(pr(s[top]) >= pr(ch)) pofx[k++]=pop(); push(ch); } } while(s[top] != '#') /* Pop from stack till empty pofx[k++]=pop(); pofx[k]='\0'; /* Make pofx as valid string */ printf("\n\nGiven Infix Expn: %s Postfix Expn: %s\n",infx,pofx); } </pre>	
--	--

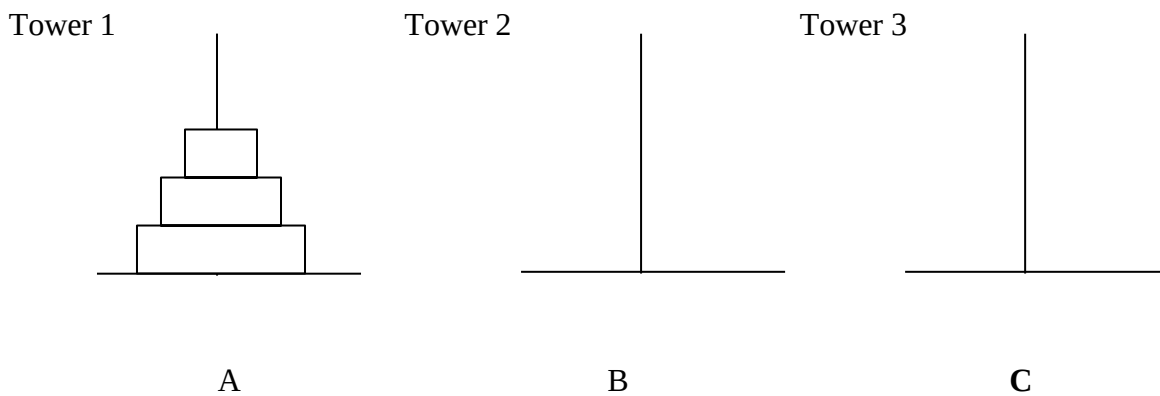
Towers of Hanoi

Towers of Hanoi can be easily implemented using recursion. Objective of the problem is moving a collection of N disks of decreasing size from one pillar to another pillar. The movement of the disk is restricted by the following rules.

Rule 1 : Only one disk could be moved at a time.

Rule 2 : No larger disk could ever reside on a pillar on top of a smaller disk.

Rule 3 : A 3rd pillar could be used as an intermediate to store one or more disks, while they were being moved from source to destination.



Initial Setup of Tower of Hanoi

Recursive Solution

N - represents the number of disks.

Step 1. If $N = 1$, move the disk from A to C.

Step 2. If $N = 2$, move the 1st disk from A to B. Then move the 2nd disk from A to C, The move the 1st disk from B to C.

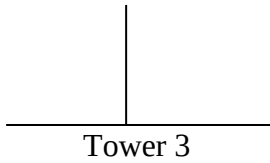
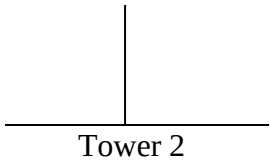
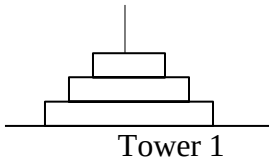
Step 3. If $N = 3$, Repeat the step (2) to move the first 2 disks from A to B using C as intermediate. Then the 3rd disk is moved from A to C. Then repeat the step (2) to move 2 disks from B to C using A as intermediate.

In general, to move N disks. Apply the recursive technique to move $N - 1$ disks from A to B using C as an intermediate. Then move the N^{th} disk from A to C. Then again apply the recursive technique to move $N - 1$ disks from B to C using A as an intermediate

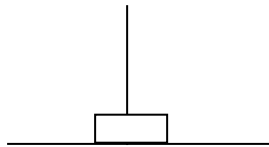
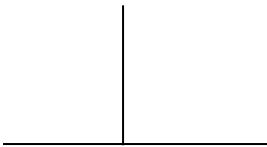
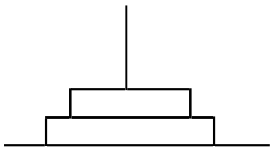
Recursive Routine for Towers of Hanoi

```
void hanoi (int n, char s, char d, char i)
{
/* n   no. of disks, s   source, d   destination i   intermediate
*/ if (n == 1)
{
print (s, d);
return;
}
else
{
hanoi (n - 1, s, i, d);
print (s, d)
hanoi (n-1, i, d, s);
return;
}
}
```

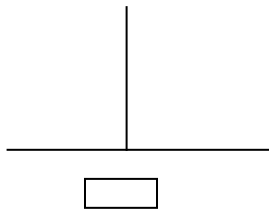
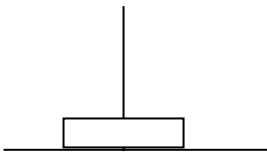
Source Pillar Intermediate Pillar Destination Pillar



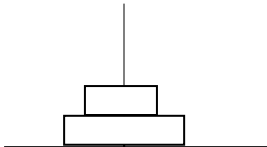
1. Move Tower1 to Tower3



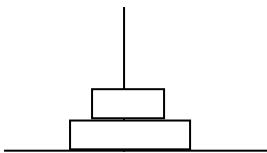
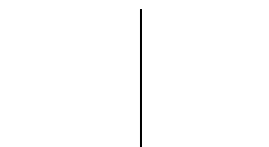
2. Move Tower1 to Tower2

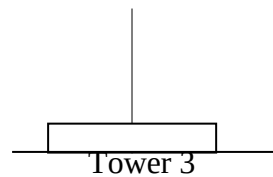
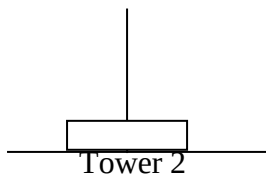
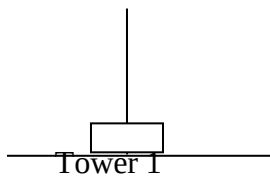


3. Move Tower 3 to Tower 2

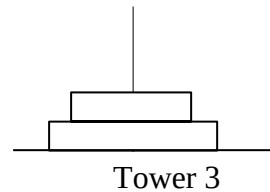
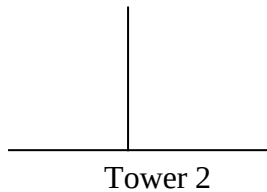
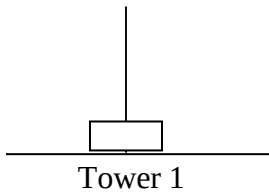


4. Move Tower 1 to Tower 3

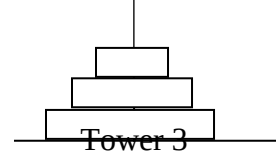
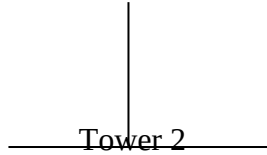
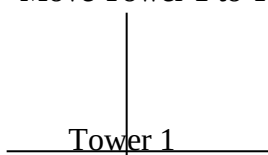




6. Move Tower 2 to Tower 3



7. Move Tower 1 to Tower 3

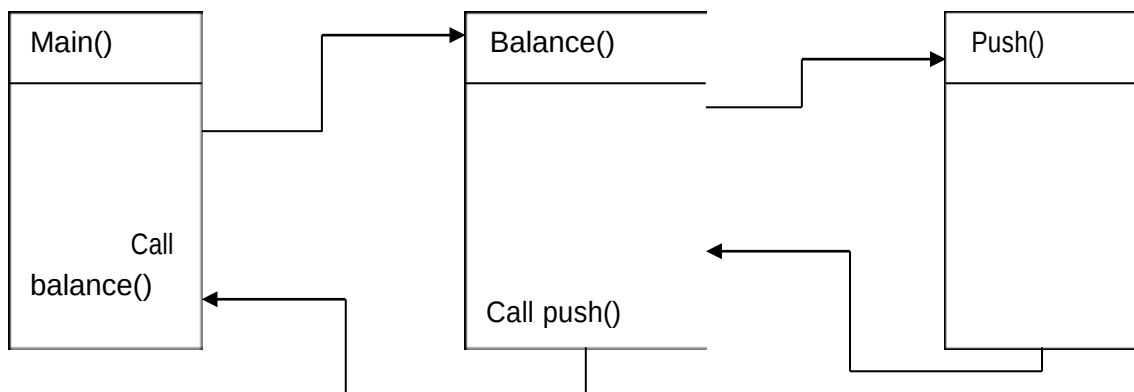


Since disks are moved from each tower in a LIFO manner, each tower may be considered as a Stack. Least Number of moves required solving the problem according to our algorithm is given by,

$$O(N)=O(N-1)+1+O(N-1)=2^N-1$$

Function Calls

When a call is made to a new function all the variables local to the calling routine need to be saved, otherwise the new function will overwrite the calling routine variables. Similarly the current location address in the routine must be saved so that the new function knows where to go after it is completed.



Recursive Function to Find Factorial

```
int fact(int n)
{
    int S;
    if(n==1)
        return(1);
    else
        S = n * fact( n - 1 );
    return(S)
}
```

Balancing the Symbols

- Compilers check the programs for errors, a lack of one symbol will cause an error.
- A Program that checks whether everything is balanced.
- Every right parenthesis should have its left parenthesis.
- Check for balancing the parenthesis brackets braces and ignore any other character.

Algorithm for balancing the symbols

Read one character at a time until it encounters the delimiter `#`.

Step 1 : - If the character is an opening symbol, push it onto the stack.

Step 2 : - If the character is a closing symbol, and if the stack is empty report an error as missing opening symbol.

Step 3 : - If it is a closing symbol and if it has corresponding opening symbol in the stack, POP it from the stack. Otherwise, report an error as mismatched symbols.

Step 4 : - At the end of file, if the stack is not empty, report an error as Missing closing symbol. Otherwise, report as balanced symbols.

E.g. Let us consider the expression $((B*B)-\{4*A*C\}/[2*A]) \#$

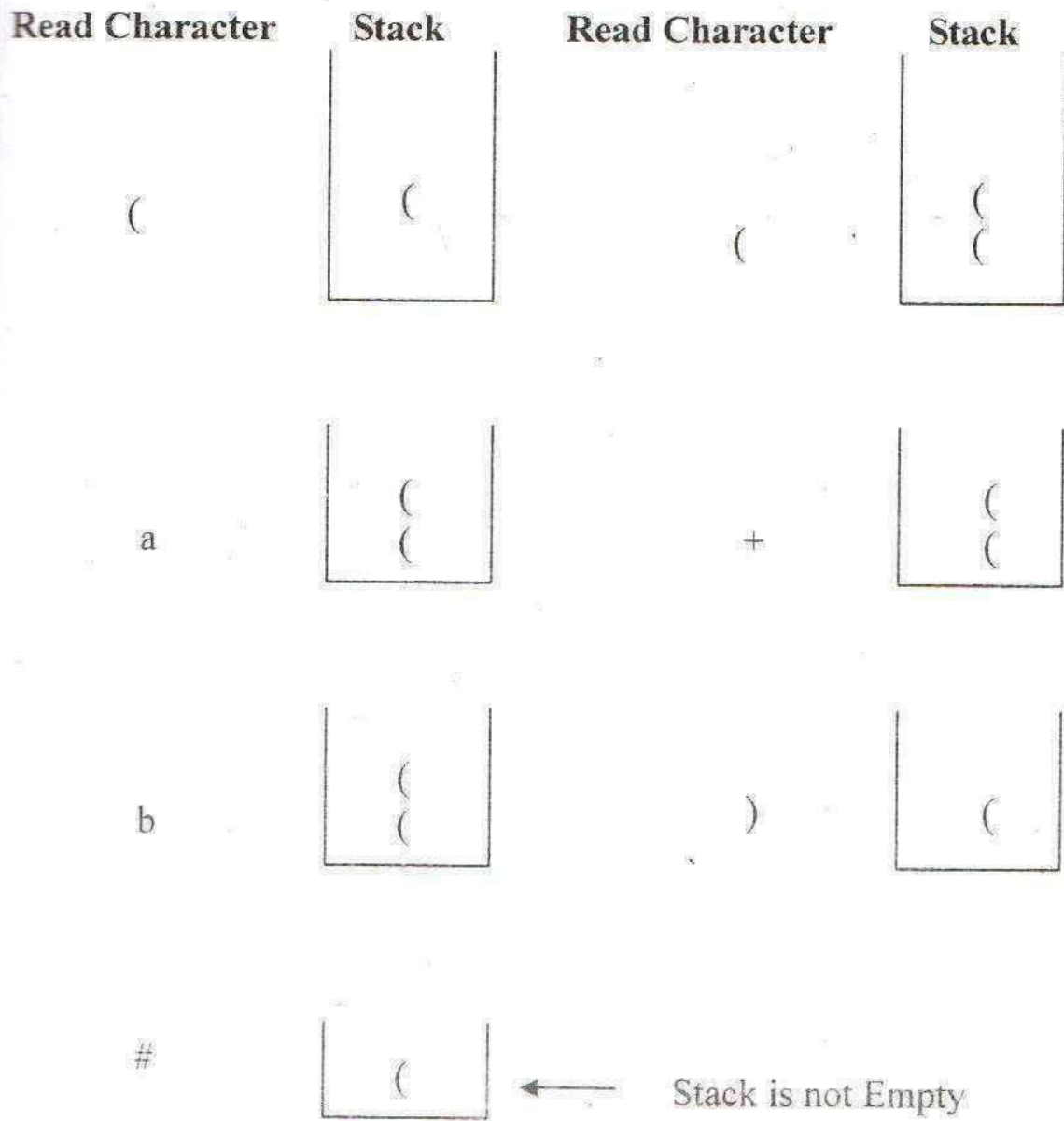
$((B*B)-\{4*A*C\}/[2*A]) \#$	
Read Character	Stack
(	(
(	((
)	 (
{	{ (
}	 (

[	[(
]	(
)	

Empty stack, hence the symbols are balanced in the given expression.

Example for unbalanced symbols:

Consider the expression $((a + b) \#$:-

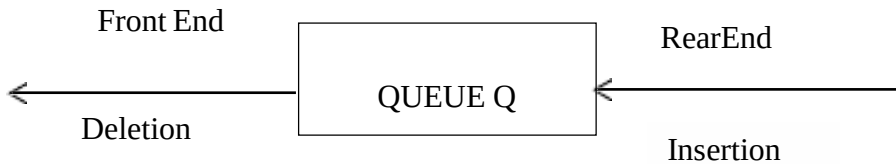


Report an error message

Illustration For Unbalanced Symbols

QUEUES:

- Queue is a Linear Data Structure that follows First in First out (FIFO) principle.
- Insertion of element is done at one end of the Queue called “**Rear** “end of the Queue.
- Deletion of element is done at other end of the Queue called “**Front** “end of the Queue.
- Example: - Waiting line in the ticket counter.



Queue Model

Front Pointer:-

It always points to the first element inserted in the Queue.

Rear Pointer:-

It always points to the last element inserted in the Queue.

For Empty Queue:-

$\text{Front (F)} = - 1$ $\text{Rear (R)} = - 1$

Operations on Queue

Fundamental operations performed on the queue are

1. EnQueue
2. DeQueue

(i) EnQueue operation:-

- It is the process of inserting a new element at the rear end of the Queue.
- For every EnQueue operation
 - Check for Full Queue
 - If the Queue is full, Insertion is not possible.
 - Otherwise, increment the rear end by 1 and then insert the element in the rear end of the Queue.

(ii) DeQueue Operation:-

- It is the process of deleting the element from the front end of the queue.
- For every DeQueue operation
 - Check for Empty queue
 - If the Queue is Empty, Deletion is not possible.
 - Otherwise, delete the first element inserted into the queue and then increment the front by 1.

Exceptional Conditions of Queue

- Queue Overflow
- Queue Underflow

(i) Queue Overflow:

- An Attempt to insert an element X at the Rear end of the Queue when the Queue is full is said to be Queue overflow.
- For every Enqueue operation, we need to check this condition.

(ii) Queue Underflow:

- An Attempt to delete an element from the Front end of the Queue when the Queue is empty is said to be Queue underflow.
- For every DeQueue operation, we need to check this condition.

Implementation of Queue

Queue can be implemented in two ways.

1. Implementation using Array (**Static Queue**)
2. Implementation using Linked List (**Dynamic Queue**)

Array Declaration of Queue:

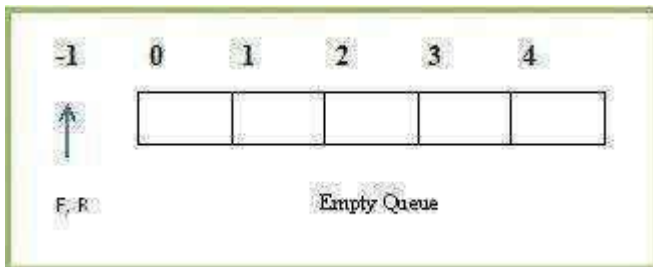
```
#define ArraySize 5
```

```
int Q [ ArraySize];
```

or

```
int Q [ 5 ];
```

Initial Configuration of Queue:



(i) Queue Empty Operation:

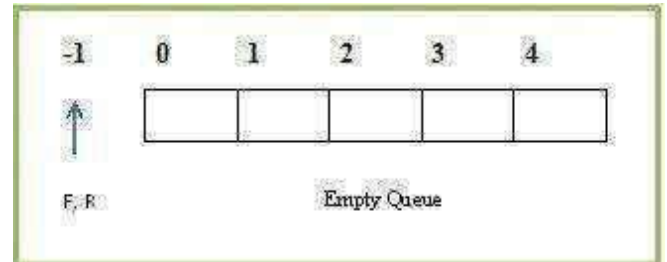
- ☐ Initially Queue is Empty.
- With Empty Queue, Front (F) and Rear (R) points to – 1.
- ☐ It is necessary to check for Empty Queue before deleting (DeQueue) an element from the Queue (Q).

Routine to check for Empty Queue

```
int IsEmpty ( Queue Q )  
{  
if( ( Front == - 1 ) && ( Rear == - 1 ) )  
return ( 1 );  
}
```

```
int IsEmpty ( Queue Q )
```

```
{  
  
if( ( Front == - 1 ) && ( Rear == - 1 ) )  
  
    return ( 1 );  
  
}
```

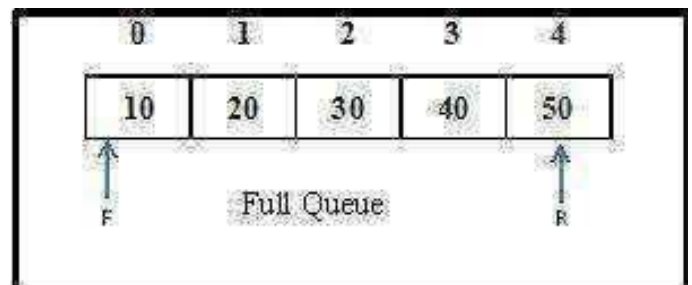


(ii) Queue Full Operation

- ❑ As we keep inserting the new elements at the Rear end of the Queue, the Queue becomes full.
- ❑ When the Queue is Full, Rear reaches its maximum Arraysize.
- ❑ For every Enqueue Operation, we need to check for full Queue condition.

Routine to check for Full Queue

```
int IsFull( Queue Q )  
  
{  
  
if ( Rear == ArraySize - 1 )  
  
    return ( 1 );  
  
}
```

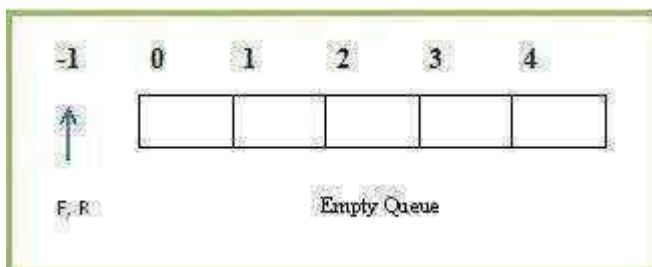
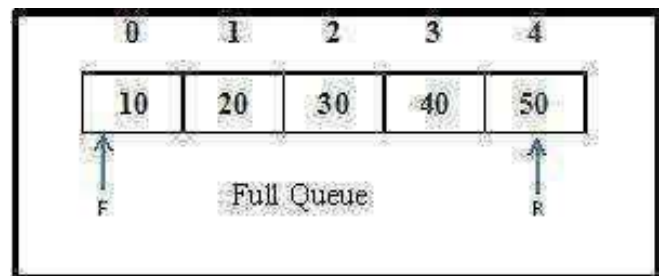


(iii) Enqueue Operation

- ❑ It is the process of inserting a new element at the Rear end of the Queue.
- ❑ It takes two parameters, Enqueue(X, Q). The elements X to be inserted at the Rear end of the Queue Q.
- ❑ Before inserting a new Element into the Queue, check for Full Queue.
- ❑ If the Queue is already Full, Insertion is not possible.
- ❑ Otherwise, Increment the Rear pointer by 1 and then insert the element X at the Rear end of the Queue.
- ❑ If the Queue is Empty, Increment both Front and Rear pointer by 1 and then insert the element X at the Rear end of the Queue.

Routine to Insert an Element in a Queue

```
void EnQueue (int X , Queue Q)
{
    if ( Rear == Arraysize - 1)
        print (" Full Queue !!!!. Insertion not
                possible");
    else if (Rear == - 1)
    {
        Front = Front + 1;
        Rear = Rear + 1;
        Q [Rear] = X;
    }
    else
    {
        Rear = Rear + 1;
        Q [Rear] = X;
    }
}
```



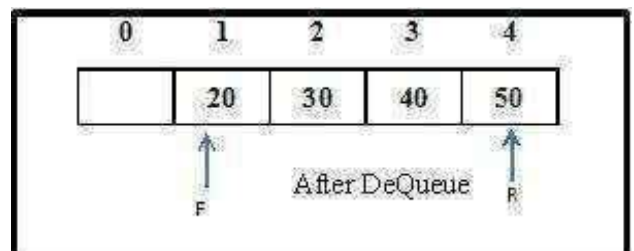
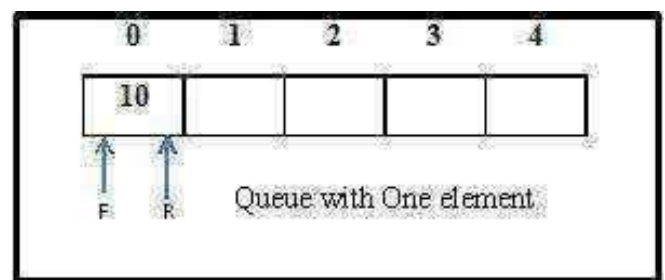
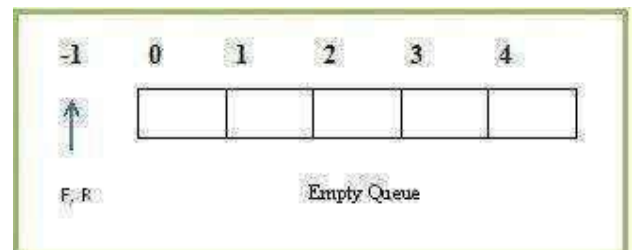


(iv) DeQueue Operation

- ☐ It is the process of deleting an element from the Front end of the Queue.
- ☐ It takes one parameter, DeQueue (Q). Always front element in the Queue will be deleted.
- ☐ Before deleting an Element from the Queue, check for Empty Queue.
- ☐ If the Queue is empty, deletion is not possible.
- ☐ If the Queue has only one element, then delete the element and represent the empty queue by updating Front = - 1 and Rear = - 1.
- ☐ If the Queue has many Elements, then delete the element in the Front and move the Front pointer to next element in the queue by incrementing Front pointer by 1.

ROUTINE FOR DEQUEUE

```
void DeQueue ( Queue Q )
{
    if ( Front == - 1 )
        print ( " Empty Queue !. Deletion not possible " );
    else if( Front == Rear )
    {
        X = Q [ Front ];
        Front = - 1;
        Rear = - 1;
    }
    else
    {
        X = Q [ Front ];
        Front = Front + 1 ;
    }
}
```



Array implementation of Queue

```
#include<stdio.h>
#include<conio.h>
#define SIZE 5
int front = - 1;
int rear = - 1;
int q[SIZE];
void insert( );
void del( );
void display( );
void main( )
{
    int choice;
    clrscr( );
    do
    {
        printf("\t Menu");
        printf("\n 1. Insert");
        printf("\n 2. Delete");
        printf("\n 3. Display ");
        printf("\n 4. Exit");

        printf("\n Enter Your Choice:");
        scanf("%d", &choice);
        switch(choice)
        {
            case 1:
                insert( );
                display( );
                break;

            case 2:
                del( );
                display( );
                break;

            case 3:
                display( );
                break;

            case 4:
                printf("End of Program... !!!!");
                exit(0);

        }
    }while(choice != 4);}

void insert( )
{
    int no;
```

```

printf("\n Enter No.:");
scanf("%d", &no);

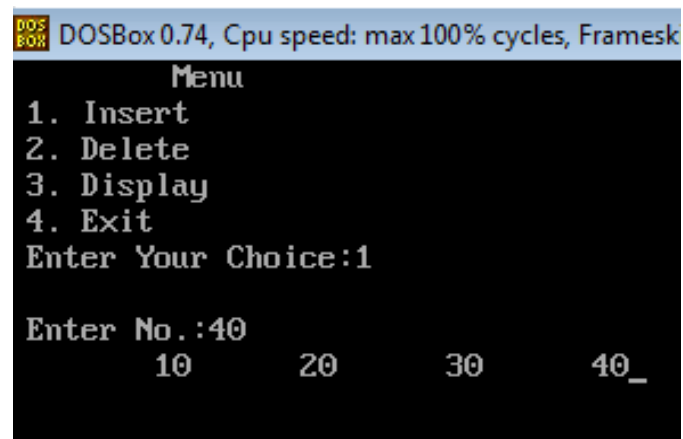
if(rear < SIZE - 1)
{
    q[++rear]=no;
    if(front == - 1)
        front=0;// front=front+1;
}
else
{
    printf("\n Queue overflow");
}}

void del()
{
    if(front == - 1)
    {
        printf("\n Queue Underflow");
        return;
    }
    else
    {
        printf("\n Deleted Item:-->%d\n", q[front]);
    }
    if(front == rear)
    {
        Front = - 1;
        Rear = - 1;
    }
    else
    {
        Front = front + 1;
    }
}

void display()
{
    int i;
    if( front == - 1)
    {
        printf("\nQueue is empty... ");
        return;
    }
    for(i = front; i<=rear; i++)
        printf("\t%d",q[i]);}

```

output



```

DOSBox 0.74, Cpu speed: max 100% cycles, Framesk
Menu
1. Insert
2. Delete
3. Display
4. Exit
Enter Your Choice:1
Enter No.:40
10      20      30      40_

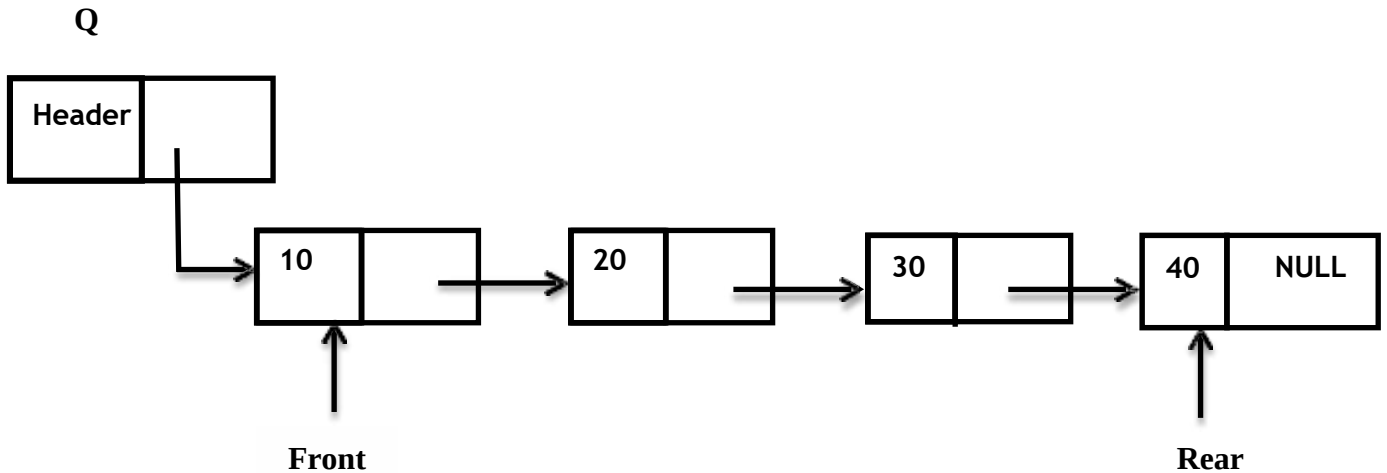
```

Linked List Implementation of Queue

- Queue is implemented using SLL (Singly Linked List) node.
- Enqueue operation is performed at the end of the Linked list and DeQueue operation is performed at the front of the Linked list.
- With Linked List implementation, for Empty queue

Front = NULL & Rear = NULL

Linked List representation of Queue with 4 elements



Declaration for Linked List Implementation of Queue ADT

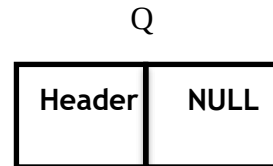
```
struct node;
typedef struct node * Queue;
typedef struct node * position;
int IsEmpty (Queue Q);
Queue CreateQueue (void);
void MakeEmpty (Queue Q);
void Enqueue (int X, Queue Q);
void Dequeue (Queue Q);
struct node
{
    int data ;
    position next;
}* Front = NULL, *Rear = NULL;
```

(i) Queue Empty Operation:

- Initially Queue is Empty.
- With Linked List implementation, Empty Queue is represented as S -> next = NULL.
- It is necessary to check for Empty Queue before deleting the front element in the Queue.

ROUTINE TO CHECK WHETHER THE QUEUE IS EMPTY

```
int IsEmpty (Queue Q)
{
    if ( Q->next == NULL)
        return (1);
}
```



Empty Queue

(ii) EnQueue Operation

- It is the process of inserting a new element at the Rear end of the Queue.
- It takes two parameters, EnQueue (int X , Queue Q). The elements X to be inserted into the Queue Q.
- Using malloc () function allocate memory for the newnode to be inserted into the Queue.
- If the Queue is Empty, the newnode to be inserted will become first and last node in the list. Hence Front and Rear points to the newnode.
- Otherwise insert the newnode in the Rear -> next and update the Rear pointer.

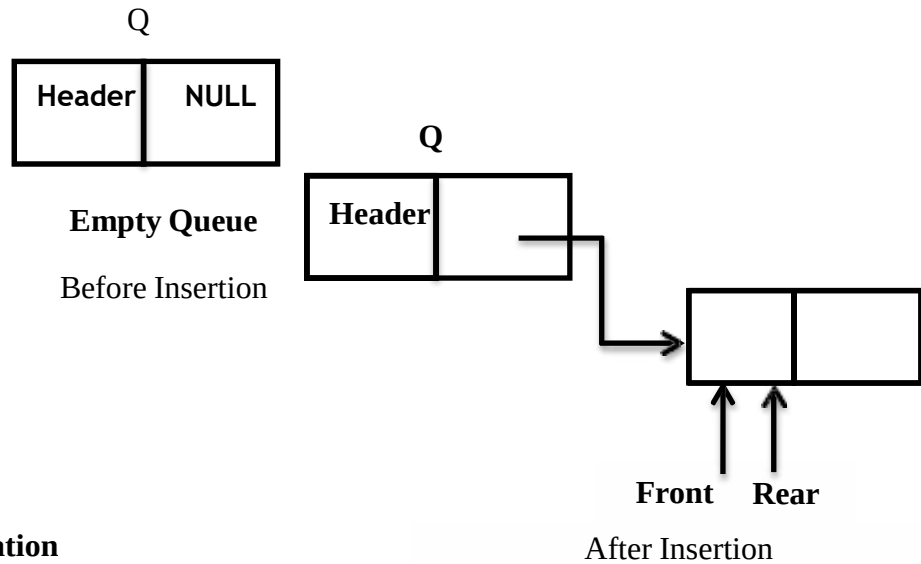
Routine to EnQueue an Element in Queue

```
void EnQueue ( int X, Queue Q )
{
    struct node *newnode;
    newnode = malloc (sizeof (struct node));
    if (Rear == NULL)
    {
        newnode->data = X;
        newnode->next = NULL;
        Q->next = newnode;
        Front = newnode;
        Rear = newnode;
    }
    else
    {
        newnode->data = X;
        newnode->next = NULL;
        Rear->next = newnode;
    }
}
```

```

Rear = newnode;
}
}

```



(iii) DeQueue Operation

It is the process of deleting the front element from the Queue.

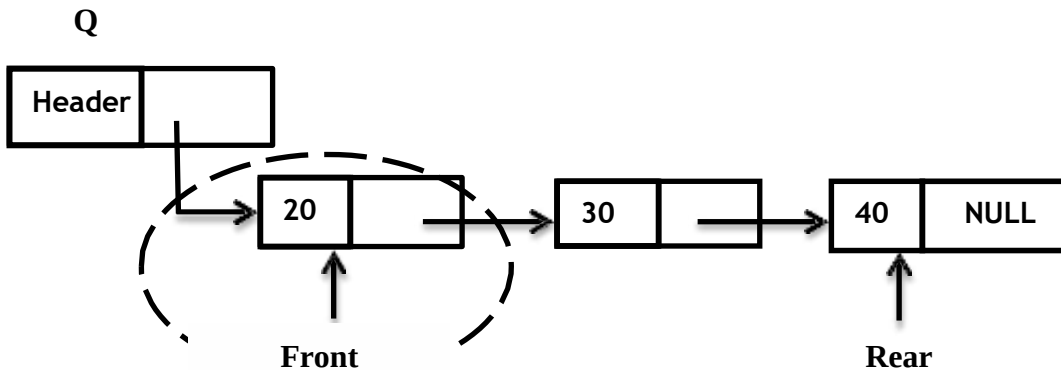
It takes one parameter, Dequeue (Queue Q). Always element in the front (i.e) element pointed by Q -> next is deleted always.

Element to be deleted is made "temp".

If the Queue is Empty, then deletion is not possible.

If the Queue has only one element, then the element is deleted and Front and Rear pointer is made NULL to represent Empty Queue.

Otherwise, Front element is deleted and the Front pointer is made to point to next node in the list. The free () function informs the compiler that the address that temp is pointing to, is unchanged but the data present in that address is now undefined.



Routine to DeQueue an Element from the Queue

```
void DeQueue ( Queue Q )
{
    struct node *temp;
    if ( Front == NULL )
        Error ("Empty Queue!!! Deletion not possible.");
    else if (Front == Rear)
    {
        temp = Front;
        Q -> next = NULL;
        Front = NULL;
        Rear = NULL;
        free ( temp );
    }
    else
    {
        temp = Front;
        Q -> next = temp -> next;
        Front = Front → Next;
        free (temp);
    }
}
```

Linked list implementation of Queue

```
#include<stdio.h>
#include<conio.h>
void enqueue();
void dequeue();
void display();
typedef struct node *position;
position front=NULL,rear=NULL,newnode,temp,p;

struct node
{
    int data;
    position next;
};
void main()
{
    int choice;
    clrscr();
    do
    {
        printf("1.Enqueue\n2.Dequeue\n3.display\n4.exit\n");
        printf("Enter your choice\n\n");
```

```

scanf("%d",&choice);
switch(choice)
{
case 1:
    enqueue();
    break;
case 2:
    dequeue();
    break;
case 3:
    display();
    break;
case 4:
    exit(0);
}
}
while(choice<5);
}

```

void enqueue()

```

{
    newnode=(struct node*)malloc(sizeof(struct node));
    printf("\n Enter the data to be enqueued\n");
    scanf("%d",&newnode->data);
    newnode->next=NULL;
    if(rear==NULL)
        front=rear=newnode;
    else {
        rear->next=newnode;
        rear=newnode;
    }
    display();
}
void dequeue()
{
    if(front==NULL)
        printf("\nEmpty queue!!!! Deletion not possible\n");
    else if(front==rear)
    {
        printf("\nFront element %d is deleted from queue!!!! now queue is empty!!!! no more deletion possible!!!!\n",front->data);
        front=rear=NULL;
    }
    else
    {
        temp=front;
        front=front->next;
        printf("\nFront element %d is deleted from queue!!!!\n",temp->data);
        free(temp);
    }
    display();
}

```

```

}
void display()
{
    p=front;
    while(p!=NULL)
    {
        printf("%d -> ",p->data);
        p=p->next;
    }
    printf("Null\n");
}

```

Output

```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
1.Enqueue
2.Dequeue
3.display
4.exit
Enter your choice
1
Enter the data to be enqueued
10
10 -> Null
1.Enqueue
2.Dequeue
3.display
4.exit
Enter your choice

```

Applications of Queue

1. Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
2. In real life, Call Center phone systems will use Queues, to hold people calling them in an order, until a service representative is free.
3. Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive, First come first served.
4. Batch processing in operating system.
5. Job scheduling Algorithms like Round Robin Algorithm uses Queue.

Drawbacks of Queue (Linear Queue)

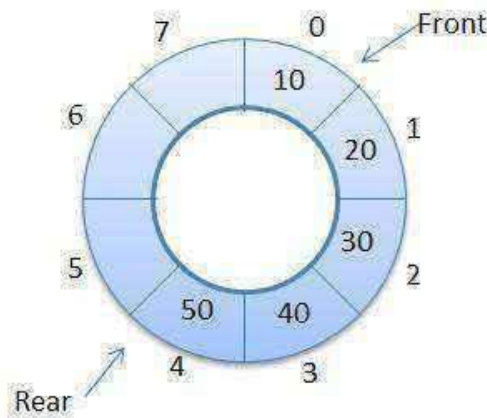
- With the array implementation of Queue, the element can be deleted logically only by moving $\text{Front} = \text{Front} + 1$.
- Here the Queue space is not utilized fully.

To overcome the drawback of this linear Queue, we use Circular Queue.

CIRCULAR QUEUE

In Circular Queue, the insertion of a new element is performed at the very first location of the queue if the last location of the queue is full, in which the first element comes just after the last element.

- A circular queue is an abstract data type that contains a collection of data which allows addition of data at the end of the queue and removal of data at the beginning of the queue.
- Circular queues have a fixed size.
- Circular queue follows FIFO principle.
- Queue items are added at the rear end and the items are deleted at front end of the circular queue
- Here the Queue space is utilized fully by inserting the element at the Front end if the rear end is full.



Operations on Circular Queue

Fundamental operations performed on the Circular Queue are

- Circular Queue Enqueue
- Circular Queue Dequeue

Formula to be used in Circular Queue

For Enqueue $\text{Rear} = (\text{Rear} + 1) \% \text{ArraySize}$

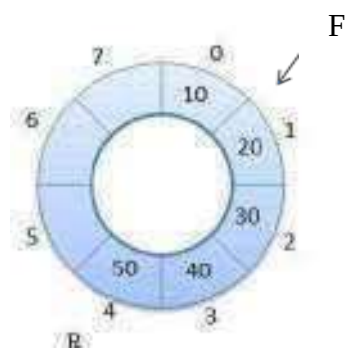
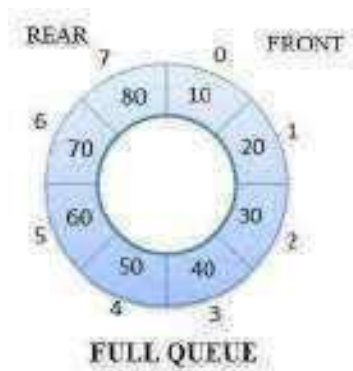
For Dequeue $\text{Front} = (\text{Front} + 1) \% \text{ArraySize}$

(i) Circular Queue Enqueue Operation

- ☐ It is same as Linear Queue EnQueue Operation (i.e) Inserting the element at the Rear end.
- ☐ First check for full Queue.
- ☐ If the circular queue is full, then insertion is not possible.
- ☐ Otherwise check for the rear end.
- ☐ If the Rear end is full, the elements start getting inserted from the Front end.

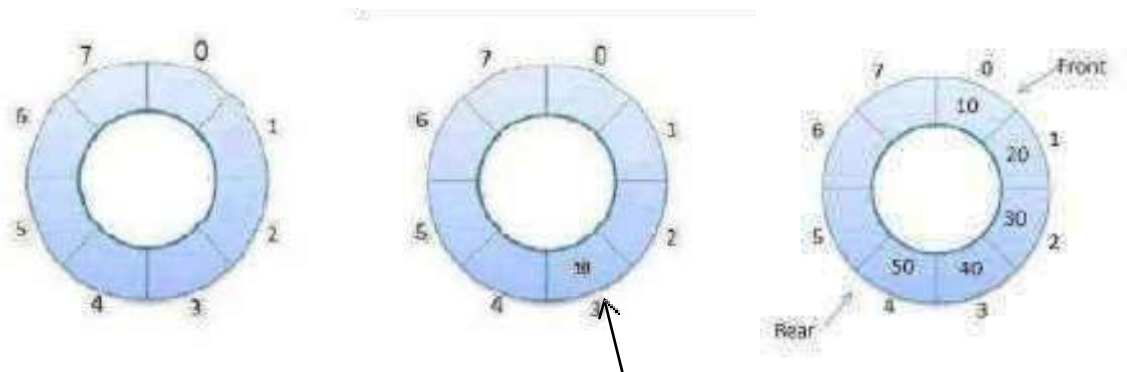
Routine to Enqueue an element in circular queue

```
void Enqueue ( int X, CircularQueue CQ )
{
    if( Front == ( Rear + 1 ) % ArraySize)
        Error( "Queue is full!!Insertion not possible" );
    else if( Rear == -1 )
    {
        Front = Front + 1;
        Rear = Rear + 1;
        CQ[ Rear ] = X;
    }
    else
    {
        Rear = ( Rear + 1 ) % Arraysize;
        CQ[ Rear ] = X;
    }
}
```



Circular Queue DeQueue Operation

- ❑ It is same as Linear Queue DeQueue operation (i.e) deleting the front element.
- ❑ First check for Empty Queue.
- ❑ If the Circular Queue is empty, then deletion is not possible.
- ❑ If the Circular Queue has only one element, then the element is deleted and Front and Rear pointer is initialized to - 1 to represent Empty Queue.
- ❑ Otherwise, Front element is deleted and the Front pointer is made to point to next element in the Circular Queue.



F= -1,R= -1

F, R

Routine To DeQueue An Element In Circular Queue

```
void DeQueue (CircularQueue CQ)
{
    if(Front== - 1)
        Empty("Empty Queue!");
    else if(Front==rear)
    {
        X=CQ[Front];
        Front=-1;
        Rear=-1;
    }
    else
    {
        X=CQ[Front];
        Front=(Front+1)%Arraysize;
    }
}
```

Implementation of Circular Queue

```
#include<stdio.h>
#include<conio.h>
#define max 3
void insert(); void delet(); void display();
int q[10],front=0,rear=-1;
void main()
{ int ch;
  clrscr();
  printf("\nCircular Queue operations\n"); printf("1.insert\n2.delete\n3.display\n4.exit\n");
  while(1)
  {
    printf("Enter your choice:");
    scanf("%d",&ch);
    switch(ch)
    {
      case 1:
        insert();      break;
      case 2:
        delet();      break;
      case 3:
        display();    break;
      case 4:
        exit();
      default:
        printf("Invalid option\n");
    }
  }
}

void insert()
{
  int x;
  if((front==0&&rear==max-1)||((front>0&&rear==front-1))
    printf("Queue is overflow\n");
  else
  {
    printf("Enter element to be insert:");
    scanf("%d",&x);
    if(rear==max-1&&front>0)
    { rear=0;
      q[rear]=x;
    }
    else
    {
      if((front==0&&rear==--1)||((rear!=front-1))
        q[++rear]=x;
```

```

    } } }
void delet()
{
    int a;
    if((front==0)&&(rear== -1))
        printf("Queue is underflow\n");
    if(front==rear)
    {
        a=q[front];
        rear=-1;
        front=0;
    }
    else if(front==max-1)
    {
        a=q[front];
        front=0;
    }
    else
        a=q[front++];
    printf("Deleted element is:%d\n",a);
}
void display()
{
    int i,j; if(front==0&&rear== -1)
        printf("Queue is underflow\n");
    if(front>rear) {
        for(i=0;i<=rear;i++)
            printf("\t%d",q[i]);
        for(j=front;j<=max-1;j++)
            printf("\t%d",q[j]);
        printf("\nrear is at %d\n",q[rear]);
        printf("\nfront is at %d\n",q[front]); }
    else
    {
        for(i=front;i<=rear;i++)
            printf("\t%d",q[i]); printf("\nrear
            is at %d\n",q[rear]); printf("\nfront
            is at %d\n",q[front]);
    }
    printf("\n");
}

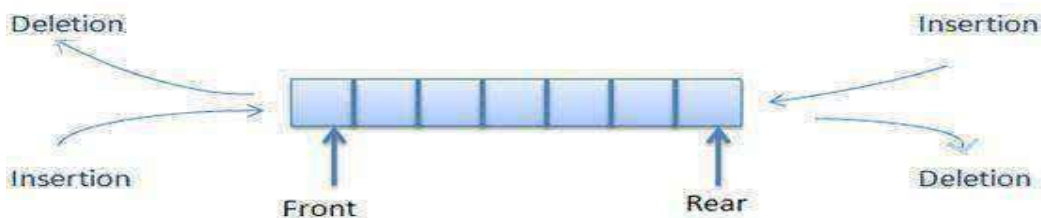
```

OUTPUT

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
1.insert ( max 3 element)
2.delete
3.display
4.exit
Enter your choice:1
Enter element to be insert:10
Enter your choice:1
Enter element to be insert:20
Enter your choice:1
Enter element to be insert:30
Enter your choice:1
Queue is overflow
Enter your choice:2
Deleted element is:10
Enter your choice:1
Enter element to be insert:40
Enter your choice:1
Queue is overflow
Enter your choice:3
    40    20    30
rear is at 40
front is at 20
Enter your choice:
```

DOUBLE-ENDED QUEUE (DEQUE)

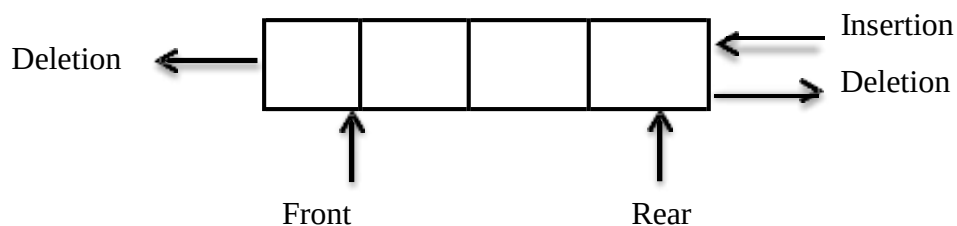
In DEQUE, insertion and deletion operations are performed at both ends of the Queue.



Exceptional Condition of DEQUE

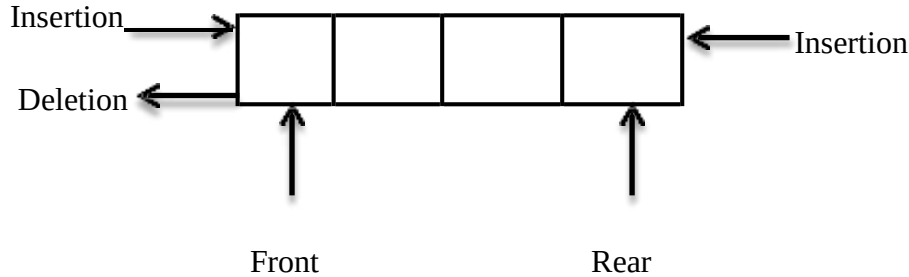
(i) Input Restricted DEQUE

Here insertion is allowed at **one end** and deletion is allowed at **both ends**.



(ii) Output Restricted DEQUE

Here insertion is allowed at **both ends** and deletion is allowed at **one end**.



Operations on DEQUE

Four cases for inserting and deleting the elements in DEQUE are

1. Insertion At Rear End [same as Linear Queue]
2. Insertion At Front End
3. Deletion At Front End [same as Linear Queue]
4. Deletion At Rear End

Insertion at the rear end

- Step 1 : Check for the overflow condition.
- Step 2 : If it is true, display that the queue is full
- Step 3 : Otherwise, If the rear and front pointers are at the initial values (-1), Increment both the pointers. Goto step 5.
- Step 4 : Increment the rear pointer
- Step 5 : Assign the value to Q[rear]

Case 1: Routine to insert an element at Rear end

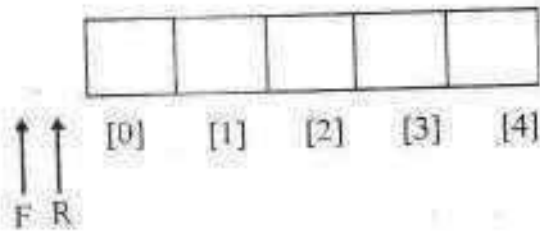
```
void Insert_Rear (int X, DEQUE DQ)
{
    if( Rear == Arraysize - 1)
        Error("Full Queue!!!! Insertion not possible");
    else if( Rear == -1)
    {
        Front = Front + 1;
        Rear = Rear + 1;
        DQ[ Rear ] = X;
    }
}
```

```

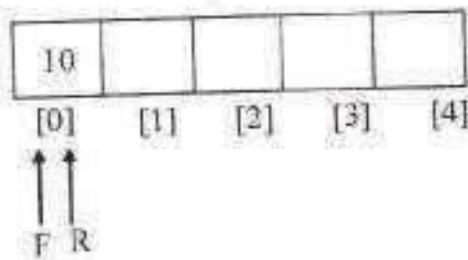
else
{
Rear = Rear + 1;
DQ[ Rear ] = X;
}
}

```

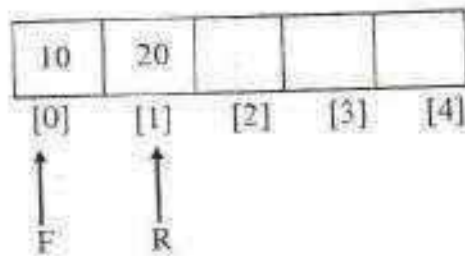
Insertion at the rear end



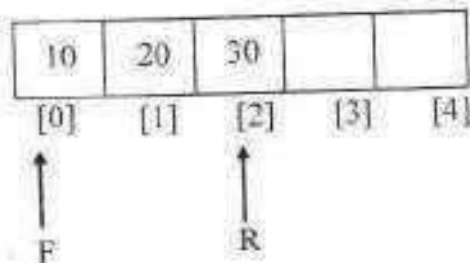
Insert_rear (10)



Insert_rear (20)



Insert_rear (30)



Insertion at front end

- Step 1 : Check the front pointer, if it is in the first position (0) then display an error message that the value cannot be inserted at the front end.
- Step 2 : Otherwise, decrement the front pointer
- Step 3 : Assign the value to Q[front]

Case 2: Routine to insert an element at Front end

```
void Insert_Front ( int X, DEQUE DQ )
{
    if( Front == 0 )
        Error("Element present in Front!!!! Insertion not possible");
    else if(Front == -1)
    {
        Front = Front + 1;
        Rear = Rear + 1;
        DQ[Front] = X;
    }
    else
    {
        Front = Front - 1;
        DQ[Front] = X;
    }
}
```

Insertion at the rear of the queue:

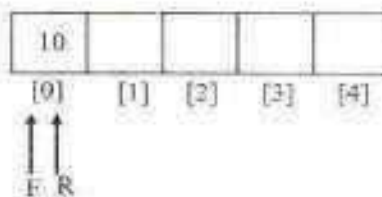
Case 1



Insertion at the rear of the queue

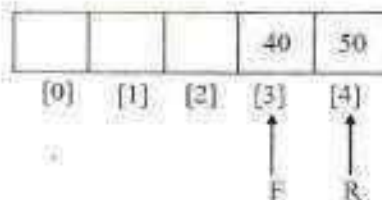


Insert_front (5)

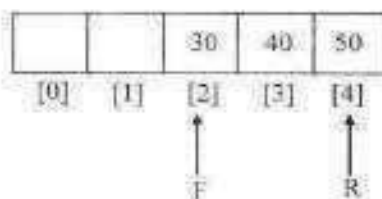


Here decrement of the front pointer may point to the initial (-1) position. So the value cannot be inserted even there is a space in the queue.

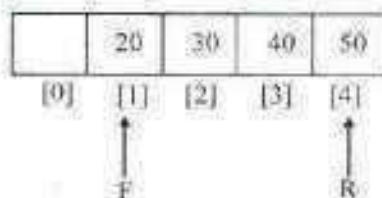
Case 2



Insert_front (30)



Insert_front (30)



Case 3: Routine to delete an element from Front end

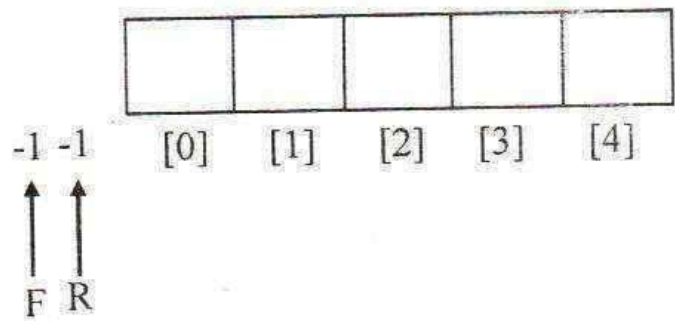
```
void Delete_Front(DEQUEUE DQ)
{
    if(Front == - 1)
        Error("Empty queue!!!! Deletion not possible");
    else if( Front == Rear )
    {
        X = DQ[ Front];
        Front = - 1;
        Rear = - 1;
    }
    else
    {
        X = DQ [ Front ];
        Front = Front + 1;
    }
}
```

Deletion from Front End :

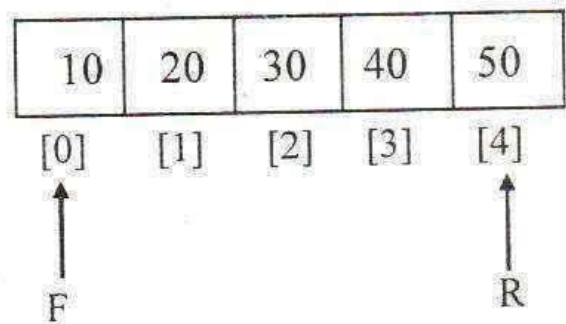
- Step 1 : Check for the underflow condition. If it is true display that the queue is empty.
- Step 2 : Otherwise, delete the element at the front position, by assigning X as Q[front]
- Step 3 : If the rear and front pointer points to the same position (ie) only one value is present, then reinitialize both the pointers.
- Step 4 : Otherwise, Increment the front pointer

Deletion from front end

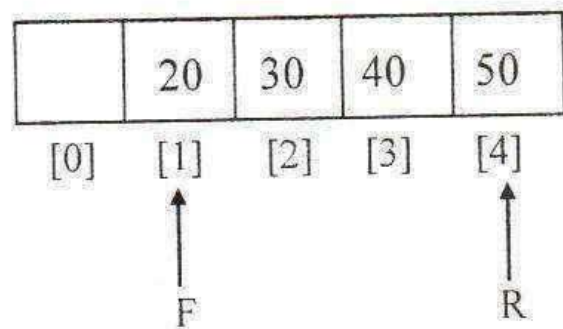
Dequeue_front()



Q - is empty



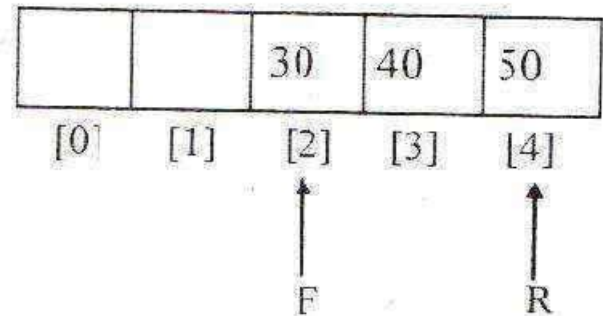
Dequeue_front()



Case 4: Routine to delete an element from Rear end

```
void Delete_Rear(DEQUE DQ)
{
    if( Rear == - 1)
        Error("Empty queue!!!! Deletion not possible");
    else if( Front == Rear )
    {
        X = DQ[ Rear ];
        Front = - 1;
        Rear = - 1;
    }
    else
    {
        X = DQ[ Rear ];
        Rear = Rear - 1;
    }
}
```

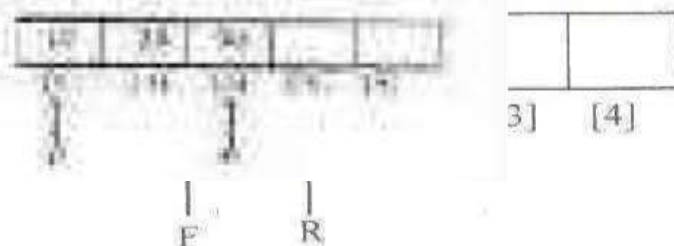
Dequeue_front()



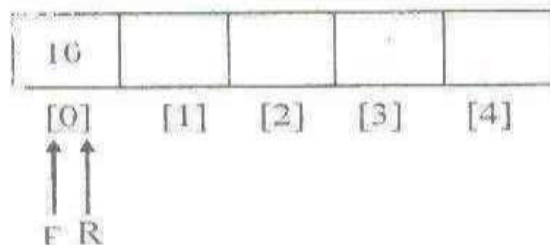
Deletion from Rear End

- Step 1 : Check the rear pointer. If it is in the initial value then display that the value cannot be deleted.
- Step 2 : Otherwise, delete element at the rear position.
- Step 3 : If the rear and front pointers are at the same position, reinitialize both the pointers.
- Step 4 : Otherwise, decrement the rear pointer.

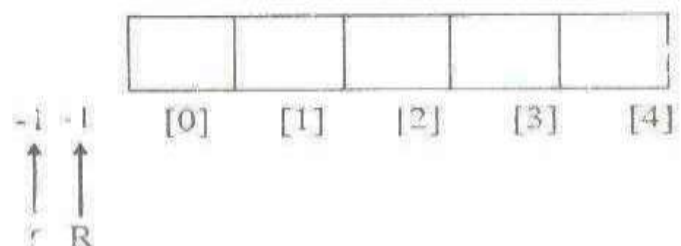
Deletion from Rear End



Deque_rrear ()



Deque_rear ()



Gate questions and answers

Stacks and Queues

1. Compute the postfix equivalent of the following infix arithmetic expression where $a+b*c+d*e\uparrow f$; where $\uparrow$ represents exponentiation. Assume normal operator precedence.

Infix expression		postfix operation	stack
1 $a+b*c+d*e\uparrow f$			
2 $+b*c+d*e\uparrow f$	a		
3 $b*c+d*e\uparrow f$	a	+	
4 $*c+d*e\uparrow f$	ab	+	
5 $c+d*e\uparrow f$	ab	+	*
6 $+d*e\uparrow f$	abc	+	*
7 $d*e\uparrow f$	abc*+	+	
8 $*e\uparrow f$	abc*+d	+	
9 $e\uparrow f$	abc*+d	+	*
10 $\uparrow f$	abc*+de	+	*
11 f	abc*+de	+	* $\uparrow$
12	abc*+def	+	* $\uparrow$

So postfix expression will be (after emptying the stack)

$abc*+def\uparrow*+$

2. Suppose one character at a time comes as an input from a string of letters. There is an option either to (i) print the incoming letter or to (ii) put the incoming letter on to a stack. Also a letter from top of the stack can be popped out at any time and printed. The total number of total distinct words that can be formed out of a string of three letters in this fashion, is
- (A)
(B)
(C)

Total no of distinct word can be 5.

because total no words starting with a would be 2. (abc ,acb)

Total no of words starting with 2 (bca,bac)

Total no of words starting with c will be 1 (cba) because to print c as first letter we have to push a and b in stack

So total no of words formed = $2+2+1$

3. The following sequence of operations is performed on a stack :

PUSH (10), PUSH (20), POP, PUSH (10), PUSH (20), POP, POP, POP, PUSH (20), POP.

The sequence of values popped out is:

- A. 20, 10, 20, 10, 20
- B. 20, 20, 10, 10, 20
- C. 10, 20, 20, 10, 20
- D. 20, 20, 10, 20, 10
- E. None of the above

[B]

String	Status of stack	Status of array(output)
Push(10)	10	
Push(20)	10 20	
Pop	10	20
Push(10)	10 10	20
Push(20)	10 10 20	20
Pop	10 10	20 20
Pop	10	20 20 10
Pop		20 20 10 10
Push(20)	20	20 20 10 10
Pop		20 20 10 10 20

4. A stack is use to pass parameters to procedures in a procedure call.

- (a) If a procedure P has two parameters as described in procedure definition:
procedure P(var x: integer; y: integer);

and if P is called by:

P(a,b)

State precisely in a sentence what is pushed onto stack for parameters a and b.

[QUESTION IS HIGHLY AMBIGUOUS]

In the generated code for the body of procedure P, how will the addressing of formal parameters y and y differ?

5. Which of the following permutation can be obtained in the output (in the same order) using a stack assuming that the input is the sequence 1, 2, 3, 4, 5 in that order?

- (A) 3, 4, 5, 1, 2
(B) 3, 4, 5, 2, 1
(C) 1, 5, 2, 3, 4
(D) 5, 4, 3, 1, 2

[B] As it can be verified as- push(1) push(2) push(3) pop push(4) pop push(5) pop pop pop

6.

The postfix expression for the infix expression

$A + B * (C + D) / F + D * E$ is

- (a) $AB + CD + * F / D + E *$
(b) $ABCD + * F / DE * ++$
(c) $A * B + CD / F * DE ++$
(d) $A + * BCD / F * DE ++$

Possible Answer:

Infix expression	Post fix operation	Operater stack
$A + B * (C + D) / F + D * E$		
$+ B * (C + D) / F + D * E$	A	
$B * (C + D) / F + D * E$	A	+
$* (C + D) / F + D * E$	AB	+
$(C + D) / F + D * E$	AB	+
$C + D) / F + D * E$	AB	+*(
$+ D) / F + D * E$	ABC	

D)/F+D*E	ABCD	
) /F+D*E	ABCD	+*(+
/F+D*E	ABCD+	+*
F+D*E	ABCD+*	+/
+D*E	ABCD+*F	+/
D*E	ABCD+*F/+	+
*E	ABCD+*F/+	+*
E	ABCD+*F/+DE	+*
	ABCD+*F/+DE*+	

7. Consider the following statements.

- i First-in-first-out types of computations are efficiently supported by STACKS.
 - ii Implementing LISTS on linked lists is more efficient than implementing LISTS on an array for almost all the basic LIST operations.
 - iii Implementing QUEUES on a circular is more efficient than implementing QUEUES
 - iv Last-in-first-out QUEUES type of computations are efficiently supported by QUEUES.
- Which of the following is correct?

(A) (ii) and (iii) are true

(B) (i) and (ii) are true

(C) (iii) and (iv) are true

(D) (ii) and (iv) are true

[A]

List perform almost all basic operation in O(1) except merging

But array will take O(n) for all operations.

Here basic operation means insert delete at given position and

Queue using linked list are efficient in using memory. They do not waste space like in array implementation .

8. Compute the postfix equivalent of the following infix expression.

$$3 * \log (x + 1) - a/2$$

$3X1 + \log^* a^2$ - (assuming log as a operator not as a function and precedence of log is greatest)

9. A queue Q containing n items and an empty stack S are given. It is required to transfer all the items from the queue to the stack, so that the item at the front of the queue is on the top of the stack, and the order of all the other items is preserved. Show how this can be done in $O(n)$ time using only a constant amount of additional storage. Note that the only operations which can be performed on the queue and stack are Delete, Insert, Push and Pop. Do not assume any implementation of the queue or stack.

It can be done as –

Step1. Delete all the nodes of queue and push all the elements in stack sequentially.

Step2. Now queue is empty then pop every element from stack and insert it into queue

Step3. Now again repeat the step 1.

10. Which of the following is essential for converting an infix expression to the postfix form efficiently?

- | | |
|--|----------------------|
| (A) An operator stack | (B) An operand stack |
| (C) An operand stack and an operator stack | (D) A parse tree |

[A] Operator stack will be used to store operators as they appear based on three rules

1. if operator 'x' is on top and operator 'y' comes which has a higher precedence than x then it will also be pushed to stack..
2. if x and y have same precedence then x will be popped out before y is pushed
3. same as rule 2 if x has a higher precedence than y.

11. A priority queue Q is used to implement a stack S that stores characters. PUSH(C) is implemented as INSERT(Q, C, K) where K is an appropriate integer key chosen by the implementation. POP is implemented as DELETETOP(Q). For a sequence of operations, the keys chosen are in

- | | |
|-------------------------------|-------------------------------|
| (A) non-increasing order | (B) non-decreasing order |
| (C) strictly increasing order | (D) strictly decreasing order |

[D] Implementing stack using priority queue require first element inserted in stack will be deleted at last, and to implement it using deletetop() operation of queue will require first element inserted must have highest priority so the key must be in decreasing order.

12. Suppose a stack implementation supports, in addition to PUSH and POP, an operation REVERSE, which reverses the order of the elements on the stack.

- (a) To implement a queue using the above stack implementation, show how to implement ENQUEUE using a single operation and DEQUEUE using a sequence of 3 operations (2)
- (b) The following postfix expression, containing single digit operands and arithmetic operators + and *, is evaluated using a stack.
 $5\ 2\ *\ 3\ 4\ +\ 5\ 2\ *\ * +$

a)

i) Implementation of enqueue is as-

Step1: push the element in stack

ii) implementation of dequeue is as-

Step1: reverse

Step2: pop

Step3:reverse

Show the contents of the stack

- i After evaluating $5\ 2\ *\ 3\ 4\ +$
- ii After evaluating $5\ 2\ *\ 3\ 4\ +\ 5\ 2$
- iii At the end of evaluation

b)

Input in postfix	Stack<- top	evaluation
5	5	
2	52	
*	10	$5*2=10$
3	10 3	
4	10 3 4	
+	10 7	$3+4=7$
5	10 7 5	
2	10 7 5 2	
*	10 7 10	$5*2=10$
*	10 70	$7*10=70$

+	80	
Null		

i) contents of stack are 10 7

ii) contents of stack are 10 7 5 2

iii) contents of stack null and the final answer is 80

13. What is the minimum number of stacks of size n required to implement a queue of size n ?
- One
 - Two
 - Three
 - Four

[B] First stack will be used to store the queue with rear at top..

We need to push element in to stack 1 in order to insert it in queue.

For deletion we will empty stack 1 in stack 2. that will put front on top so we can pop from stack 2 in order to effect deletion.

14. Let S be a stack of size $n \geq 1$. Starting with the empty stack, suppose we Push the first n natural numbers in sequence, and then perform n Pop operations. Assume that Push and POP operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For $m \geq 1$, define the stack-life of m as the time elapsed from the end of Push(m) to the start of the Pop operation that removes m from S . The average stack-life of an element of this stack is

- $n(X + Y)$
- $3Y + 2X$
- $N(X + Y) - X$
- $Y + 2X$

[A]

Push1() and pop1() are push and pop operations in 1st stack

Push2() and pop2() are push and pop operations in 2nd stack

Using 1st stack we can insert value same as queue (push1())

And for deletion we have to pop all the elements of 1st stack into push it into 2nd stack and then pop one element form 2nd stack.(pop1()push2()).....until 1st stack is emptied then pop2()...for desired number of deletion)

15. The following sequence of operations is performed on a stack :

PUSH (10), PUSH (20), POP, PUSH (10), PUSH (20), POP, POP, POP, PUSH (20), POP.

The sequence of values popped out is:

- F. 20, 10, 20, 10, 20
- G. 20, 20, 10, 10, 20
- H. 10, 20, 20, 10, 20
- I. 20, 20, 10, 20, 10
- J. None of the above

[G]

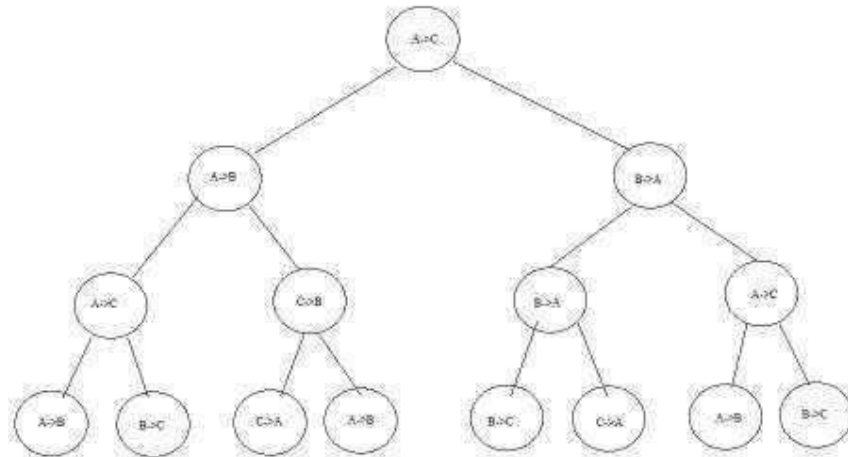
String	Status of stack	Status of array(output)
Push(10)	10	
Push(20)	10 20	
Pop	10	20
Push(10)	10 10	20
Push(20)	10 10 20	20
Pop	10 10	20 20
Pop	10	20 20 10
Pop		20 20 10 10
Push(20)	20	20 20 10 10
Pop		20 20 10 10 20

16. Consider three pegs A, B, C and four disks of different sizes. Initially, the four disks are stacked on peg A, in order of decreasing size. The task is to move all the disks from peg A to peg C with the help of peg B. the moves are to be made under the following constraints:

- [i] In each step, exactly one disk is moved from one peg to another.
- [ii] A disk cannot be placed on another disk of smaller size. If we denote the movement of a disk from one peg to another by $y \rightarrow z$, where y, z are A, B or C, then represent the sequence of the minimum number

of moves to accomplish this as a binary tree with node labels of the form $(y \rightarrow y)$ such that the in-order traversal of the tree gives the correct sequence of the moves..

If there are n disks, derive the formula for the total number of moves required in terms of n



For one disk the steps involved is $A \rightarrow C$ ie 1 step

For two disks the steps involved are $A \rightarrow B, A \rightarrow C, B \rightarrow C$ ie 3 steps

For three disks the steps involved are $A \rightarrow C, A \rightarrow B, C \rightarrow B, A \rightarrow C, B \rightarrow A, B \rightarrow C, A \rightarrow C$ ie 7 steps

For 4 disks the steps involved are $A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, C \rightarrow A, C \rightarrow B, A \rightarrow B, A \rightarrow C, B \rightarrow C, B \rightarrow A, C \rightarrow A, B \rightarrow A, A \rightarrow B, A \rightarrow C, B \rightarrow C$ ie 15 steps

So for 1 disk $2^1 - 1 = 1$

2 disks $2^2 - 1 = 3$

3 disks $2^3 - 1 = 7$

4 disks $2^4 - 1 = 15$

.....

.....

n disks $2^n - 1$ (by induction)

17. A stack is use to pass parameters to procedures in a procedure call.

(b) If a procedure P has two parameters as described in procedure definition:
 procedure $P(\text{var } x: \text{integer}; y: \text{integer});$

and if P is called by:

P(a,b)

State precisely in a sentence what is pushed onto stack for parameters a and b.

In the generated code for the body of procedure P, how will the addressing of formal parameters y and y differ?

[QUESTION IS HIGHLY AMBIGUOUS]

18. Which of the following permutation can be obtained in the output (in the same order) using a stack assuming that the input is the sequence 1, 2, 3, 4, 5 in that order?

- (E) 3, 4, 5, 1, 2
- (F) 3, 4, 5, 2, 1
- (G) 1, 5, 2, 3, 4
- (H) 5, 4, 3, 1, 2

[F] As it can be verified as- push(1) push(2) push(3) pop push(4) pop push(5) pop pop pop

19. Consider the following C program:

```
#include<stdio.h>

#define EOF -1

void push(int);          /* Push the argument on the stack */
int pop(void);           /* pop the top of the stack */
void flagError();

int main( )
{
    int c, m, n, r;

    while ((c = getchar( )) != EOF)
    {
        if (isdigit(c))
            push(c)

        else if (c == '+' || (c == '*'))
        {
            m = pop( );
            n = pop( );
            are = (c == '+') ? n + m : n*m;
            push(r);
        }

        else if (c != ' ')
```

```

        flagError( );

    }

    printf(“%c”, pop( ));

}

```

What is the output of the program for the following input?

5 2 * 3 3 2 + * +

(A) 15 (B) 25 (C) 30 (D) 150

[B] Sequence of operations will be

1. push(5)
2. push(2)
3. m = 2; n=5 ; r=5*2
4. push(r) /*push(10)*/
5. push(3)
6. push(3)
7. push(2)
8. m=2; n= 3; r=3+2
9. push(r) /* push(5)*/
10. r=5*3
11. push(r) /*push(15)*/
12. m=15; n = 10; r=15+10
13. push(r)
14. printf(“%d”,pop());

Which will print 25

20. Suppose you are given an implementation of a queue of integers. The operations that can be performed on the queue are:

isEmpty(Q)	– returns true if the queue is empty, false otherwise.
delete(Q)	– deletes the element at the front of the queue and returns its value.
insert(Q, i)	– inserts the integer i at the rear of the queue.

Consider the following function:

```

void f(queue Q)
{
    int i;

```

```

        if(!isEmpty (Q)) {

            i = delete(Q);

            f(Q)

            insert(Q, i);

        }

    }

```

What operation is performed by the above function f?

- (A) Leaves the queue Q unchanged
- (B) Reverse the order of elements in the queue Q
- (C) Deletes the element at the front of the queue Q and inserts it at the rear keeping the other elements in the same order
- (D) Empties the queue Q.

[B] Due to recursion the last element deleted will be the first to be inserted so element that was at rear will be at front in new queue and the second last element will be at second position hence the queue will be reversed.

21. Let w be the minimum weight among all edge weights in an undirected connected graph. Let e be a specific edge of weight w . Which of the following is **FALSE**?

- (A) There is a minimum spanning tree containing e .
- (B) If e is not in a minimum spanning tree T , then in the cycle formed by adding e to T , all edges have the same weight.
- (C) Every minimum spanning tree has an edge of weight w .
- (D) e is present in every minimum spanning tree.

[D]

22. Consider the following C function:

```

int f(int n)

{

    static int are = 0;

    If (n <= 0) return 1;

    If (n > 3)

        { r = n;

            return f (n - 2) + 2;

        }

    return f(n - 1) + r;
}

```

}

What is the value of $f(5)$?

- (A) 3 (B) 7 (C) 9 (D) 18

[D]

$f(5)$ will return $f(3)+2$

$f(3)$ will return $f(2)+5$

$f(2)$ will return $f(1)+5$

$f(1)$ will return $f(0)+5$

$f(0)$ will return 1

$f(1)$ will return 6

$f(2)$ will return 11

$f(3)$ will return 16

$f(5)$ will return 18

23. The following postfix expression with single digit operands is evaluated using a stack:

$8\ 2\ 3\ ^\wedge / 2\ 3\ * + 5\ 1\ * -$

Note that $^\wedge$ is the exponentiation operator. The top two elements of the stack after the first $*$ is evaluated are:

- (A) 6, 1 (B) 5, 7 (C) 3, 2 (D) 1, 5

[A]

Symbols read so far	Operand stack
8	8
8 2	8 2
8 2 3	8 2 3
8 2 3 [^]	8 8
8 2 3 [^] /	1
8 2 3 [^] / 2	1 2

8 2 3 [√] /2 3	1 2 3
8 2 3 [√] /2 3 *	1 6

24. An implementation of queue Q, using stacks S1 and S2 is given below:

```

void insert (Q, x) {
    push (S1, x);
}

void delete (Q) {
    if (stack-empty (S2)) then
        if (stack-empty (S1)) then {
            print ( " Q is empty" );
            return;
        }
        else while (! (stack-empty (S1))) then {
            x = pop (S1);
            push (S2, x);
        }
        x = pop (S2);
    }
}

```

Let n insert and m ($\leq n$) delete operations be performed in an arbitrary order on an empty queue Q. let x and y be the number of push and pop operations performed respectively in the process. Which one of the following is true for all m and n ?

- (A) $n + m \leq x < 2n$ and $2m \leq y \leq n + m$ (B) $n + m \leq x < 2n$ and $2m \leq y \leq 2n$
 (C) $2m \leq x < 2n$ and $2m \leq y \leq n + m$ (D) $2m \leq x < 2n$ and $2m \leq y \leq 2n$

[A]

Extreme case will be when all the elements are inserted and then deleted

For each insert we have a push operation

And for first delete we have n push operations if n elements are in stack, so $n+m \leq x$ in worst case

And no of pushes is also $2n : n$ for n insert and n for first delete

So $(n+m) \leq x < 2n$

Also n pops for first delete and $m-1$ one pops for $m-1$ deletes so $y = n+m-1$ in extreme case $y < n+m-1$

25. A function f defined on stacks of integers satisfies the following properties.

$$f(\phi) = 0 \text{ and}$$

$$f(\text{push}(S, i)) = \max(f(S), 0) + i \text{ for all stacks } S \text{ and integers } i.$$

If a stack S contains the integers 2, -3, 2, -1, 2 in order from bottom to top, what is $f(S)$?

(A) 6

(B) 4

(C) 3

(D) 2

[C] Answer is 3

Step1: top $s=2$

Step2: top $s=-1$

Step3: top $s=2$

Step4: top $s=1$

Step5: top $s=3$

26. Assume that the operators $+$, $-$, $\times$ are left associative and $\wedge$ is right associative. The order of precedence (from highest to lowest) is $\wedge$, $\times$, $+$, $-$. The postfix expression corresponding to the infix expression $a+b \times c-d \wedge e \wedge f$ is

- (A) $abc \times + def \wedge \wedge -$
(B) $abc \times + de \wedge f \wedge -$
(C) $ab+c \times d-e \wedge f \wedge$
(D) $-+a \times bc \wedge \wedge def$

[A]

Infix string	Stack	Postfix array
a		a
+	+	a
b	+	ab
*	+	ab

c	+	abc
-	-	abc*+
d	-	abc*+d
^	-^	abc*+d
e	-^	abc*+de
^	-^^ (not popped because of right associative)	abc*+de
f	-^^	abc*+def
	-^	abc*+def^
	-	abc*+def^^
		abc*+def^^-

27. The best data structure to check whether an arithmetic expression has balanced parenthesis is a

- (A) Queue
- (B) Stack
- (C) Tree
- (D) List

[B] We can put every opening parenthesis in stack and pop every time we get a closed one. In the end if we get stack empty then we had balanced parenthesis else not.